

SQLP 과목 III 튜닝 스타일 모의문제 · 4회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

오라클 데이터베이스 버퍼 캐시의 내부 관리 — LRU 리스트, 버퍼 상태(더티·프리), DBWR의 역할 — 에 대한 설명으로 가장 적절하지 않은 것은?

- ① 서버 프로세스가 변경한 더티 버퍼는 커밋되는 순간 그 서버 프로세스가 직접 디스크에 기록한 뒤 프리 버퍼로 전환되며, 그해야 커밋이 완료되므로 커밋 시점에 해당 블록의 데이터파일 반영이 함께 끝난다.
- ② 오라클은 자주 참조되는 블록일수록 캐시에 오래 남도록 블록을 만질 때마다 터치 카운트를 올리고, 이 카운트를 반영해 LRU 리스트에서 밀려날(재사용될) 대상을 고른다.
- ③ 서버 프로세스가 읽으려는 블록이 캐시에 없으면(캐시 미스) LRU 리스트를 뒤져 프리 버퍼를 확보하고 그 자리에 디스크 블록을 적재하며, 프리 버퍼를 찾기 어려우면 DBWR를 깨워 더티 버퍼를 디스크로 내려 공간을 비운다.
- ④ 더티 버퍼는 변경됐으나 아직 데이터파일에 반영되지 않은 버퍼이고 프리 버퍼는 디스크와 내용이 같아 즉시 덮어써도 되는 버퍼이며, DBWR의 기록이 끝나 내용이 디스크와 일치하게 되면 더티 버퍼는 프리 버퍼로 상태가 바뀐다.

2

대량 스캔 시의 두 가지 I/O 경로 — 버퍼 캐시(SGA)를 경유하는 Multiblock I/O와 그것을 우회하는 Direct Path Read — 와 그때 관측되는 대기 이벤트에 대한 설명으로 가장 적절한 것은?

- ① Full Table Scan의 Multiblock I/O든 병렬 쿼리의 Direct Path Read든 읽어 들인 블록은 버퍼 캐시에 적재되어 다른 세션이 재활용할 수 있고, 대기 이벤트도 둘 다 db file scattered read로 동일하게 관측된다.
- ② Direct Path Read는 서버 프로세스가 디스크 블록을 버퍼 캐시에 먼저 올린 뒤 자신의 PGA로 복사하는 방식이라, 캐시에 해당 블록의 더티 버퍼가 남아 있으면 그것을 그대로 읽어 디스크 접근을 건너뛴다.
- ③ Direct Path Read는 읽은 블록을 버퍼 캐시(SGA)를 거치지 않고 프로세스의 PGA로 곧바로 적재하며, 이때 관측되는 대기 이벤트는 db file scattered read가 아니라 direct path read다.
- ④ 병렬 쿼리로 대량 세그먼트를 훑을 때는 블록을 캐시에 두고 공유하는 편이 유리하므로, 오라클은 병렬 Full Scan을 Direct Path Read가 아니라 버퍼 캐시를 경유하는 Multiblock I/O로 처리하는 것을 기본으로 삼는다.

사원 약 50,000명의 급여명세를 생성하는 PL/SQL 배치이다. 루프 안에서 사원마다 ... WHERE 사원번호 = 10023 처럼 사원번호를 리터럴로 조립한 SQL을 실행해 급여를 계산하고 명세 1건씩을 INSERT한다. 아래는 이 배치의 TKPROF 결과를 비재귀(non-recursive)와 재귀(recursive) 통계로 분리한 것이다. 이 트레이스에 대한 옳은 진단은? (단, 옵티마이저 통계는 최신이고 산출된 결과 10만 행은 검증상 정확하며, 사원 SQL은 사원번호를 바인드 변수가 아닌 리터럴로 조립해 실행했다.)

아 래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.002	0.002	0	0	0	0
Execute	1	0.001	0.001	0	0	0	0
Fetch	2	0.010	0.020	0	40	0	1
Total	4	0.013	0.023	0	40	0	1
Misses in library cache during parse: 1							
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	50000	12.400	18.600	0	0	0	0
Execute	50000	9.200	14.100	800	180000	50000	50000
Fetch	98000	6.100	9.300	1200	240000	0	50000
Total	198000	27.700	42.000	2000	420000	50000	100000
Misses in library cache during parse: 50000							

- ① 재귀 Call이 198,000회로 전체의 약 100%인데, 이는 처리한 데이터량(10만 행)이 과도하기 때문이다. 급여 대상 사원 범위를 좁혀 재귀 Call을 줄이면 42초가 해소된다.
- ② 재귀 Parse 50,000회에 라이브러리 캐시 Miss가 50,000회(100%)로 매 실행이 하드파싱이며, 파싱이 재귀 CPU 27.7초의 약 45%(12.4초)를 소비한다. 사원번호 리터럴을 바인드 변수로 바꿔 커서를 공유하면 하드파싱과 재귀 파싱 부하가 함께 줄어든다. 재귀 Call 수는 내부·파싱 SQL의 지표일 뿐 결과 데이터량과는 별개다.
- ③ 재귀 Call이 비재귀(4회)의 수만 배에 달하므로 산출된 급여 결과의 정확성을 신뢰할 수 없고, 재귀 SQL을 수동으로 걷어내 결과를 다시 검증하는 것이 우선이다.
- ④ 재귀 Call·파싱 부하가 크다는 것은 옵티마이저가 나쁜 실행계획을 골랐다는 신호이므로, 힌트로 실행계획을 고정하면 재귀 Call과 42초가 함께 줄어든다.

4

애플리케이션이 주식주문 테이블(약 1,800만 건)에서 특정 기간의 체결 내역 약 360,000건을 읽어 화면·파일로 내려받는 SQL의 TKPROF 결과이다. 스캔 자체는 가볍게 끝나는데 응답이 28초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 디스크·CPU 자원 경합은 없으며, 애플리케이션의 배열 인출(fetch array) 크기는 기본값을 그대로 사용했다.)

아래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.003	0.003	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	3601	3.200	28.800	900	180500	0	360000
Total	3603	3.203	28.803	900	180500	0	360000
Rows	Row Source Operation						
360000	TABLE ACCESS FULL 주식주문 (cr=180500 pr=900 pw=0 time=4120000 us)						

- ① 물리 읽기 Disk 900이 Elapsed 28.8초를 만든 근본 원인이므로, 주식주문 테이블을 더 빠른 스토리지로 옮기면 응답이 개선된다.
- ② Full Scan이 논리 읽기 cr 180,500 블록을 소비하는 것이 병목이므로, 체결일시에 인덱스를 만들어 논리 읽기를 줄이면 28.8초가 해소된다.
- ③ $\text{ArraySize} = 360,000 \div (3,601 - 1) = 100$ 으로 배열 크기가 작아 Fetch가 3,601회 발생했고, 스캔 시간 4.1초와 Fetch Elapsed 28.8초의 약 24.7초 간극은 인출 왕복(round-trip) 대기다. 배열 크기를 키우면(예: 1,000) Fetch가 약 361회로 줄어 왕복 대기가 함께 감소한다.
- ④ BCHR이 약 99.5%로 높고 Row Source의 스캔 시간이 4.1초뿐이므로 DB 작업은 이미 최적이며, 남은 24초대는 손댈 수 없는 네트워크 지연이라 SQL 차원에서 줄일 수 없다.

5

예상(추정) 실행계획을 조회하는 DBMS_XPLAN.DISPLAY와 실제 수행된 커서의 계획을 조회하는 DBMS_XPLAN.DISPLAY_CURSOR, 그리고 바인드 변수 때문에 둘이 갈릴 수 있는 상황에 대한 설명으로 가장 적절한 것은?

- ① DBMS_XPLAN.DISPLAY_CURSOR는 아직 수행하지 않은 문장의 예상 계획을 PLAN_TABLE에서 읽어 오는 함수이므로, EXPLAIN PLAN을 선행하지 않고도 DISPLAY와 서로 바꿔 써서 예상 계획을 확인할 수 있다.
- ② DBMS_XPLAN.DISPLAY는 EXPLAIN PLAN이 PLAN_TABLE에 적재한 예상 계획을 보여 주고, DBMS_XPLAN.DISPLAY_CURSOR는 라이브러리 캐시에 올라온 실제 수행된 커서의 계획을 보여 준다. 바인드 변수가 있으면 EXPLAIN PLAN은 그 값을 엿보지(bind peeking) 못해, 실제 수행 시의 계획과 서로 달라질 수 있다.
- ③ EXPLAIN PLAN은 대상 문장을 실제로 파싱·수행하면서 바인드 변수 값을 반영해 계획을 세우므로, DISPLAY로 조회한 예상 계획이 곧 실제 커서의 계획과 같다.
- ④ AUTOTRACE의 SET AUTOTRACE TRACEONLY EXPLAIN으로 출력되는 계획은 라이브러리 캐시의 실제 커서에서 가져온 것이라, 실측 행 수가 포함된 실제 실행계획이며 DISPLAY_CURSOR로 본 계획과 같다.

6

예금거래 테이블(약 800만 건)에는 세 칼럼 결합 인덱스 예금거래_IX = (예금계좌번호, 거래일자, 거래구분)이 있다. 예금계좌명을 함께 얻기 위해 예금계좌 테이블(기본키 예금계좌_PK = 예금계좌번호)과 조인하는 다음 SQL의 실행계획이다.

```
SELECT d.거래번호, d.거래일자, d.거래금액, a.예금계좌명
FROM 예금거래 d, 예금계좌 a
WHERE d.예금계좌번호 = '110-204-778'
AND d.거래일자 >= DATE '2026-06-01'
AND d.거래일자 < DATE '2026-07-01'
AND d.거래구분 = '이체'
AND a.예금계좌번호 = d.예금계좌번호;
```

이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(212	940	61100)
1	0	NESTED LOOPS	(212	940	61100)
2	1	TABLE ACCESS BY INDEX ROWID 예금거래	(208	940	42300)
3	2	INDEX RANGE SCAN 예금거래_IX	(14	940	)
4	1	TABLE ACCESS BY INDEX ROWID 예금계좌	(1	1	20)
5	4	INDEX UNIQUE SCAN 예금계좌_PK	(0	1	)

- ① 예금계좌번호·거래일자·거래구분 세 칼럼 모두 INDEX RANGE SCAN의 스캔 시작·종료 지점을 결정하는 액세스 조건이므로, 세 조건이 함께 스캔 범위를 최소로 좁힌다.
- ② 예금계좌번호 등치와 거래일자 범위까지가 스캔 범위를 결정하는 액세스 조건이고, 그 뒤 칼럼인 거래구분은 범위 조건 뒤에 놓여 스캔 범위를 더 좁히지 못한 채 인덱스 필터로만 처리된다.
- ③ 스캔 범위를 결정하는 액세스 조건은 선두 칼럼 예금계좌번호 하나뿐이고, 거래일자 범위는 스캔 범위와 무관하게 인덱스 필터로 처리된다.
- ④ 거래구분은 인덱스에 없어 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 걸리지며, 그 때문에 TABLE ACCESS의 Card가 INDEX RANGE SCAN보다 작아진다.

7

참고이동 테이블(약 800만 건, 세그먼트 약 80,000 블록)에서 참고이동_IX 인덱스를 경유해 특정 기간의 이동 내역을 조회한 SQL의 트레이스이다. 이 트레이스에 대한 옳은 진단은? (단, 옵티마이저 통계는 최신이고 인덱스는 Unusable 상태가 아니며, 이 SQL의 SELECT 목록은 참고이동일자, 이동수량 두 컬럼뿐이다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.005	0.005	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	5001	4.900	31.500	40000	121600	0	500000
Total	5003	4.905	31.505	40000	121600	0	500000
Rows	Row Source Operation						
500000	TABLE ACCESS BY INDEX ROWID 참고이동 (cr=121600 pr=40000 pw=0 time=31200000 us)						
500000	INDEX RANGE SCAN 참고이동_IX (cr=1600 pr=200 pw=0 time=780000 us)						

- ① 물리 읽기 40,000이 논리 읽기 121,600의 약 33%로 BCHR이 약 67%이므로 버퍼 캐시 부족이 근본 원인이고, 캐시를 키우면 이 SQL이 해결된다.
- ② 랜덤 읽기 121,600 > Full Scan 80,000이라 손익분기점을 넘었으므로 Full Scan 전환만이 유일한 해법이며, 테이블이 커서 커버링 인덱스로는 개선할 수 없다.
- ③ ROWID당 블록 읽기가 약 0.24로 1보다 작으니 클러스터링 팩터는 최상급이고, 31초 대기는 인덱스 리프 블록 분할(cr=1,600) 때문이다.
- ④ 테이블 랜덤 액세스가 ROWID 50만 건에 블록 I/O 12만을 써 CF가 테이블 블록 수(80,000)의 약 24배로 나뉘고, 그 랜덤 블록(121,600-1,600=120,000)이 cr의 98.7%를 차지한다. SELECT가 두 컬럼뿐이라 커버링 인덱스로 TABLE ACCESS BY INDEX ROWID를 없애면 cr이 약 1,600 수준으로 떨어져 Full Scan(80,000)보다도 유리하다.

8

Index Full Scan과 Index Fast Full Scan, 그리고 테이블 액세스 없이 인덱스만 읽고 끝나는 스캔에 대한 설명으로 가장 적절하지 않은 것은?

- ① Index Full Scan은 인덱스 리프 블록을 논리적 연결 순서를 따라 한 블록씩 Single Block I/O로 끝까지 훑으므로, 인덱스 키 순서로 정렬된 결과를 얻을 수 있어 ORDER BY를 대체해 소트를 생략할 수 있다.
- ② Index Fast Full Scan은 인덱스 리프 블록의 논리적 연결 순서를 따라 읽으므로 결과가 인덱스 키 순서로 정렬되어 나오고, 그 덕분에 ORDER BY를 소트 없이 대체할 수 있다.
- ③ 쿼리가 필요로 하는 컬럼이 인덱스 안에 모두 들어 있으면, 어느 방식으로 스캔하든 테이블 액세스(TABLE ACCESS BY INDEX ROWID)를 생략하고 인덱스만 읽어 결과를 낼 수 있다.
- ④ Index Fast Full Scan은 인덱스 세그먼트 전체를 물리적 저장 순서대로 Multiblock I/O로 읽어 대량 스캔에 유리하고, 병렬로도 수행할 수 있다.

9

관측소별 기상 자료를 담은 관측실적 테이블에 결합 인덱스 관측_IX = (관측소코드, 관측일시, 항목코드)(모두 오름차순)가 있다. 인덱스의 정렬 순서와 ORDER BY의 소트 생략, 결합 인덱스 컬럼 순서에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스는 키를 오름차순으로 저장하므로 ORDER BY 관측소코드 ASC는 소트를 생략할 수 있지만, ORDER BY 관측소코드 DESC는 저장 순서와 반대여서 인덱스로는 정렬을 대체할 수 없고 별도의 소트 연산이 뒤따른다.
- ② 관측소코드 = :c로 선두 컬럼을 한 점에 고정한 뒤 ORDER BY 관측일시로 정렬하면, 그 구간 안에서 관측일시 순서가 살아 있어 소트를 생략할 수 있다.
- ③ ORDER BY 관측소코드, 관측일시처럼 정렬 컬럼의 순서와 방향이 인덱스 구성과 일치하면, 옵티마이저는 인덱스를 순서대로 읽는 것만으로 정렬 결과를 내어 소트 연산을 생략할 수 있다.
- ④ 가운데 컬럼(관측일시)에 조건이 없고 선두(관측소코드)와 세 번째(항목코드)에만 조건이 걸리면, 항목코드는 스캔 시작·끝점을 좁히지 못하고 인덱스 필터로만 처리된다.

10

최근 한 달 안에 마일리지를 적립한 적이 있는 가입자만 골라내기 위해, 가입자 테이블(약 90만 건)에 대해 적립내역 테이블(약 6,200만 건)로 EXISTS 서브쿼리를 건 SQL의 실행계획이다. 상관 조건은 가입자ID 등치(=)이고, 적립일자 범위는 서브쿼리 안의 조건이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아래

```
SELECT m.가입자ID, m.가입자명, m.등급
FROM 가입자 m
WHERE EXISTS ( SELECT 1
                FROM 적립내역 p
                WHERE p.가입자ID = m.가입자ID
                  AND p.적립일자 >= DATE '2026-06-18' )
ORDER BY m.등급;
```

Id	Operation	Name
0	SELECT STATEMENT	
1	SORT ORDER BY	
* 2	HASH JOIN SEMI	
3	TABLE ACCESS FULL	가입자
4	TABLE ACCESS BY INDEX ROWID	적립내역
* 5	INDEX RANGE SCAN	적립내역_IX1

- ① EXISTS가 세미 조인으로 변환돼 HASH JOIN SEMI로 수행되며, 가입자 한 건에 대해 적립내역에서 조건을 만족하는 행을 하나라도 찾으면 더 찾지 않고 그 가입자를 결과에 포함한다. 그래서 적립내역에 여러 건이 매칭돼도 가입자 행이 중복 증식되지 않는다.
- ② HASH JOIN SEMI는 조인 결과에서 중복 행을 걸러내는 SORT UNIQUE·DISTINCT와 같은 연산이므로, 적립내역의 중복 가입자ID를 먼저 제거한 뒤 조인한다.
- ③ 이 플랜은 NOT EXISTS가 변환된 HASH JOIN ANTI이며, 적립내역에 매칭이 하나도 없는 가입자만 반환한다.
- ④ 세미 조인은 일반 조인과 같아서, 매칭되는 적립내역 건수만큼 가입자 행이 늘어나 가입자가 여러 번 중복 출력된다.

11

해시 조인(Hash Join)의 Hash Area 사용과 처리 방식(one-pass·multi-pass)에 대한 설명으로 가장 적절한 것은?

- ① 해시 조인은 Build Input과 Probe Input을 각각 조인 키로 정렬한 뒤 양쪽을 맞대어 병합하므로, 두 입력이 미리 정렬돼 있으면 정렬 단계를 줄일 수 있다.
- ② Build Input으로 만든 해시 테이블이 Hash Area에 통째로 들어가면, Probe 단계에서도 Probe Input 전체를 Hash Area에 함께 올려 두 해시 테이블을 메모리에서 대조한다.
- ③ Build Input이 Hash Area에 다 담기지 못하면 옵티마이저는 두 입력을 같은 기준으로 여러 파티션으로 나눠 일부 파티션을 Temp 테이블스페이스에 기록했다가 파티션 단위로 조인하며, 이 과정을 one-pass 또는 multi-pass로 수행한다.
- ④ 해시 조인은 조인 키를 해시 버킷에 고르게 분산해 매칭하므로 등치(=)뿐 아니라 부등호·범위 조인에서도 성립하며, 그런 범위 조인에서 NL 조인보다 유리하다.

12

오라클 옵티마이저 모드(ALL_ROWS·FIRST_ROWS)와 최적화 목표에 대한 설명으로 가장 적절하지 않은 것은?

- ① FIRST_ROWS(n)는 첫 행을 빨리 반환하도록 최적화하므로, 결과 전체를 끝까지 읽어 내는 총 수행 시간도 ALL_ROWS로 얻는 계획보다 함께 짧아진다.
- ② FIRST_ROWS(n)는 앞쪽 n건을 빨리 돌려주는 응답 시간을 목표로 삼는 모드로, 첫 행을 일찍 내보내기 위해 정렬을 인덱스로 대체하는 계획을 선호할 수 있다.
- ③ RBO는 통계와 무관하게 미리 정해진 규칙 순위로 실행계획을 정하던 옛 방식으로, 지금은 사실상 폐기되어 비용 기반의 CBO가 기본 옵티마이저다.
- ④ ALL_ROWS는 결과 집합을 끝까지 가져오는 데 드는 총비용을 줄이도록 계획을 세우는 처리량 중심 모드로, 전체범위 최적화를 지향한다.

13

옵티마이저의 쿼리 변환 중 조건절 Pushing과 조건절 이행(Transitive Predicate Generation)에 대한 설명으로 가장 적절한 것은?

- ① 조건절 Pushing은 바깥 조건을 인라인 뷰 안으로 밀어 넣는 변환이므로, 그 조건을 받은 인라인 뷰는 바깥 쿼리와 병합되지 못하고 독립된 블록으로 분리되어 수행된다.
- ② 조인 조건 A.key = B.key와 상수 조건 B.key = 10이 함께 있으면, 옵티마이저는 A.key = 10이라는 새 조건을 이끌어 내어 A 쪽 인덱스를 액세스 조건으로 쓸 수 있게 한다.
- ③ 조건절 이행으로 A.key = 10이 유도되면 원래의 조인 조건 A.key = B.key는 중복이 되어 실행계획에서 제거되고, 두 테이블은 조인 없이 각자 상수 조건으로만 걸러진다.
- ④ 조건절 Pushing과 조건절 이행은 둘 다 조건을 옮겨 다루는 변환이므로, 조건을 안쪽으로 미는 하나의 기법을 상황에 따라 달리 부른 이름이다.

14

상담센터 통계 화면이 상담유형별 건수를 동적 SQL(Dynamic SQL)로 조회한다. 개발자는 아래 두 방식을 두고 고민 중이다. 방식 A는 상담유형 값을 문자열로 결합(concatenation) 해 SQL을 조립하고, 방식 B는 바인드 변수로 값을 실행 시점에 결합한다. 상담유형 3종('INBOUND'·'OUTBOUND'·'VOC')을 각각 1회씩 조회한 직후 라이브러리 캐시(v\$sql)를 요약한 결과가 함께 주어졌다. 커서 공유와 동적 SQL에 대한 설명으로 가장 적절하지 않은 것은?

아래

```

-- 방식 A : 문자열 연결(리터럴 결합)
v_sql := 'SELECT COUNT(*) FROM 상담이력 '
        || 'WHERE 상담유형 = ''' || v_type || ''';
EXECUTE IMMEDIATE v_sql INTO v_cnt;

-- 방식 B : 바인드 변수
v_sql := 'SELECT COUNT(*) FROM 상담이력 WHERE 상담유형 = :1';
EXECUTE IMMEDIATE v_sql INTO v_cnt USING v_type;

[ 라이브러리 캐시(v$sql) 요약 - 상담유형 3종을 각각 1회씩 조회 ]
SQL_ID          SQL_TEXT (선두 요약)                                EXECUTIONS  LOADS
-----
방식 A (문자열 연결)
a7m2k9x4p1qwz   ...WHERE 상담유형 = 'INBOUND'                        1            1
f3n8t5v2r6bdc   ...WHERE 상담유형 = 'OUTBOUND'                        1            1
k9w4h1s7m0xje   ...WHERE 상담유형 = 'VOC'                              1            1
방식 B (바인드 변수)
p5c8d2b6n0tly   ...WHERE 상담유형 = :1                                3            1

```

- ① 방식 A는 상담유형 값마다 SQL 텍스트가 달라져, 값의 종류만큼(여기서는 3개) 서로 다른 SQL_ID로 각각 하드파싱(LOADS 1)되고 라이브러리 캐시에 커서가 누적된다.
- ② 방식 A는 사용자 입력을 문자열로 이어 붙여 SQL을 조립하므로 v_type에 악의적 문자열이 섞이면 SQL Injection에 노출되고, 값이 다양할수록 하드파싱이 늘어 라이브러리 캐시 부하도 커진다.
- ③ 방식 B는 :1 바인드로 텍스트가 고정되어 SQL_ID 하나(p5c8...)에 EXECUTIONS만 3으로 늘고, 값이 실행 시점에 결합되므로 SQL Injection에도 상대적으로 안전하다.
- ④ 방식 A도 상수 리터럴만 다를 뿐 구조가 같으므로, 오라클 기본 설정에서 옵티마이저가 리터럴을 바인드 변수로 자동 치환해 세 조회를 하나의 커서로 공유한다.

15

수질 관측소가 하루치 원시 측정값 약 4,000만 건을 스테이징 테이블(수질측정_적재)에서 본 테이블(수질측정)로 적재하는 배치이다. 개발자는 아래처럼 세션에 병렬 DML을 활성화한 뒤 APPEND PARALLEL 힌트로 대량 INSERT를 수행하고, 커밋하기 전에 같은 세션에서 방금 적재한 행 수를 확인하려 한다. 이 배치의 병렬 DML·Direct Path Insert 동작에 대한 설명으로 가장 적절하지 않은 것은?

아래

```

ALTER SESSION ENABLE PARALLEL DML;

INSERT /*+ APPEND PARALLEL(t, 4) */ INTO 수질측정 t
SELECT * FROM 수질측정_적재;

-- 아직 COMMIT 하지 않은 상태에서
SELECT COUNT(*) FROM 수질측정;      -- 같은 트랜잭션에서 재조회 시도

```

- ① ALTER SESSION ENABLE PARALLEL DML을 먼저 실행해야 INSERT 문의 PARALLEL 힌트가 병렬 DML로 동작한다. 세션에서 활성화하지 않으면 INSERT 자체는 직렬로 처리될 수 있다.
- ② APPEND 힌트로 Direct Path Insert가 되어 기존 세그먼트의 프리리스트를 거치지 않고 HWM(고수위) 위 새 블록에 기록하며, 병렬 INSERT는 각 PX 서버가 자기 임시 세그먼트에 적재한 뒤 HWM을 올려 병합한다.
- ③ 대상 테이블이 NOLOGGING이고 Direct Path로 적재하면 데이터에 대한 redo 생성이 최소화되어 적재는 빨라지지만, 미디어 복구가 불가능해질 수 있어 적재 직후 백업이 필요할 수 있다.
- ④ 병렬 DML로 INSERT한 직후, 같은 트랜잭션 안에서 COMMIT 없이 곧바로 SELECT COUNT(*) FROM 수질측정으로 방금 넣은 행 수를 조회해 확인할 수 있다.

세금계산서 테이블은 발행일자(DATE)를 기준으로 분기별 RANGE 파티셔닝되어 있고, 그 위에 세 종류의 파티션 인덱스가 있다(아래 DDL). **ix_loc_pfx**는 선두에 파티션 키를 포함한 로컬(prefixed), **ix_loc_npfx**는 파티션 키를 포함하지 않은 로컬(nonprefixed), **ix_glob**은 계산서번호로 별도 RANGE 분할한 글로벌 인덱스이다. 이 파티션 인덱스들의 스캔·유지보수 특성에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
CREATE TABLE 세금계산서 (
  계산서번호 NUMBER,
  발행일자 DATE,
  공급자번호 VARCHAR2(10),
  거래처번호 VARCHAR2(10),
  공급가액 NUMBER
)
PARTITION BY RANGE (발행일자) (
  PARTITION p_2026q1 VALUES LESS THAN (DATE '2026-04-01'),
  PARTITION p_2026q2 VALUES LESS THAN (DATE '2026-07-01'),
  PARTITION p_2026q3 VALUES LESS THAN (DATE '2026-10-01'),
  PARTITION p_2026q4 VALUES LESS THAN (DATE '2027-01-01'),
  PARTITION p_max VALUES LESS THAN (MAXVALUE)
);

-- ix_loc_pfx : 로컬, 선두에 파티션 키(발행일자) 포함 → prefixed
CREATE INDEX ix_loc_pfx ON 세금계산서 (발행일자, 공급자번호) LOCAL;

-- ix_loc_npfx : 로컬, 선두 칼럼이 파티션 키가 아님 → nonprefixed
CREATE INDEX ix_loc_npfx ON 세금계산서 (거래처번호) LOCAL;

-- ix_glob : 글로벌, 계산서번호로 별도 RANGE 분할
CREATE INDEX ix_glob ON 세금계산서 (계산서번호)
GLOBAL PARTITION BY RANGE (계산서번호) (
  PARTITION g1 VALUES LESS THAN (100000),
  PARTITION g2 VALUES LESS THAN (200000),
  PARTITION g3 VALUES LESS THAN (MAXVALUE)
);
```

- ① ix_loc_pfx는 로컬 프리픽스드 인덱스라, WHERE에 발행일자 조건이 주어진다면 파티션 프루닝으로 해당 데이터 파티션에 대응하는 인덱스 파티션만 탐색한다.
- ② ix_loc_npfx는 선두 칼럼이 파티션 키(발행일자)가 아니므로, 거래처번호만으로 조회하면 대상 행이 어느 데이터 파티션에 있는지 알 수 없어 모든 인덱스 파티션을 훑어야 한다.
- ③ ix_loc_npfx도 로컬 인덱스이므로, 발행일자 조건이 없어도 거래처번호 조회 시 단일 인덱스 파티션만 접근하면 되어 프루닝 효과는 프리픽스드 로컬 인덱스와 같다.
- ④ 특정 분기 파티션을 ALTER TABLE 세금계산서 DROP PARTITION p_2026q1으로 제거하면 딸린 로컬 인덱스 파티션도 함께 사라져 유지보수가 간단하지만, 글로벌 인덱스 ix_glob은 UNUSABLE로 표시되어 REBUILD(또는 UPDATE INDEXES)가 필요할 수 있다.

17

처방전 테이블(약 2억 2천만 건)과 조제내역 테이블(약 8,900만 건)을 처방ID로 병렬 해시 조인하는 야간 정산 배치 SQL의 실행계획이다. 두 테이블은 파티셔닝돼 있지 않고, 어느 쪽도 조인 키로 미리 나뉘어 있지 않다. 이 병렬 실행계획은 두 테이블을 병렬 서버(Slave) 사이에 어떻게 분배하고 있는가? 가장 정확히 설명한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10002			Q1,02	P->S
* 3	HASH JOIN				Q1,02	PCWP
4	PX RECEIVE				Q1,02	PCWP
5	PX SEND HASH	:TQ10000			Q1,00	P->P
6	PX BLOCK ITERATOR				Q1,00	PCWC
7	TABLE ACCESS FULL	처방전			Q1,00	PCWP
8	PX RECEIVE				Q1,02	PCWP
9	PX SEND HASH	:TQ10001			Q1,01	P->P
10	PX BLOCK ITERATOR				Q1,01	PCWC
11	TABLE ACCESS FULL	조제내역			Q1,01	PCWP

- ① 두 테이블이 처방ID로 동일하게 파티셔닝돼 있어, 각 서버가 대응 파티션 쌍만 PX SEND 없이 조인하는 파티션 와이즈 조인이다.
- ② 소형 조제내역을 병렬 서버 전체에 복제(broadcast)하고, 대형 처방전은 재분배 없이 각 서버가 자기 조각만 스캔하는 broadcast-none 분배다.
- ③ 처방전과 조제내역을 양쪽 다 조인 키(처방ID) 해시로 재분배한다(hash-hash). 두 테이블 모두 대형이라 소형을 전체 복제하는 broadcast는 각 서버에 큰 테이블을 통째로 반복 복제해 비용이 과다하므로, 각 입력을 PX SEND HASH로 한 번씩만 재분배해 같은 해시 버킷끼리 조인한다.
- ④ 대형 처방전을 처방ID 해시로 재분배하고, 소형 조제내역을 병렬 서버 전체에 복제(broadcast)하는 hash-broadcast 분배다.

18

소트(Sort) 오퍼레이션의 종류와 인덱스를 이용한 소트 대체에 대한 설명으로 가장 적절한 것은?

- ① ORDER BY·GROUP BY처럼 정렬 순서에 기대는 오퍼레이션은 인덱스의 정렬된 순서를 이용하면 소트를 생략할 수 있지만, GROUP BY 없는 전체 집계(SORT AGGREGATE)는 값을 훑어 누적할 뿐 순서에 기대지 않아 인덱스로 대체되는 소트가 아니다.
- ② 소트 오퍼레이션은 종류를 가리지 않고, 정렬 순서를 제공하는 인덱스만 있으면 실행계획에서 사라지므로 SORT UNIQUE·SORT GROUP BY·SORT AGGREGATE가 인덱스로 대체된다.
- ③ 정렬 대상이 Sort Area(PGA work area)에 담기지 못해 디스크 소트가 일어나면, 그 소트는 종류를 불문하고 실행계획에 SORT UNIQUE 오퍼레이션으로 표시된다.
- ④ 인덱스가 정렬된 순서를 제공한다는 성질에 근거하면, SORT AGGREGATE로 처리되는 전체 건 집계(예: 전체 COUNT·SUM)도 인덱스 정렬 순서를 이용하면 소트 단계가 사라진다.

19

SQL Server에서 병상배정 테이블의 병동 'W3' 행(초기 잔여병상 = 8)을 두 세션이 아래 순서로 다룬다. TX2는 같은 조건의 SELECT를 두 번(t1·t3) 수행하고, 그 사이 t2에서 TX1이 잔여병상을 3 줄이고 곧바로 커밋한다. TX2의 격리수준은 기본값 READ COMMITTED이다. TX2가 t1·t3에서 읽는 값 (가)·(나)와 그때 발생하는 현상에 대한 진단으로 가장 적절한 것은? (단, RCSI는 꺼져 있어 락 기반 READ COMMITTED로 동작하며, 발문에 없는 힌트·격리수준 조정은 하지 않았다.)

아래

시각	TX1 (병상 배정 반영)	TX2 (현황 재조회, READ COMMITTED)
t1		SELECT 잔여병상 FROM 병상배정 WHERE 병동='W3'; → (가) 읽음
t2	UPDATE 병상배정 SET 잔여병상 = 잔여병상 - 3 WHERE 병동='W3'; COMMIT; (→ 5)	
t3		SELECT 잔여병상 FROM 병상배정 WHERE 병동='W3'; → (나) 읽음

초기 잔여병상(W3) = 8

- ① READ COMMITTED에서도 TX2의 첫 SELECT가 잡은 공유(S) 락이 트랜잭션 끝까지 유지되므로 (가)=(나)=8이고, 반복 읽기가 보장된다.
- ② TX2가 t3에서 아직 커밋되지 않은 값을 미리 읽어 (나)=5가 된 Dirty Read이며, 이 현상은 READ UNCOMMITTED에서만 나타난다.
- ③ 두 SELECT가 모두 커밋된 값만 읽었다는 점이 이상현상이 없음을 뒷받침하며, 최종적으로 (가)=(나)=5이다.
- ④ (가)=8, (나)=5로 같은 조건의 재조회 결과가 달라진 Non-Repeatable Read이며, TX2를 REPEATABLE READ로 올리면 첫 읽기의 공유 락이 유지되어 TX1의 UPDATE가 대기하므로 (나)도 8로 반복 읽기가 보장된다.

20

렌탈계약 테이블을 두 세션이 엇갈려 수정 중이다. 세션 30은 계약번호 1001 행을 먼저 SELECT ... FOR UPDATE로 잠근 뒤 계약번호 2002 행을 갱신하려 하고, 세션 52는 반대로 2002 행을 먼저 잠근 뒤 1001 행을 갱신하려 한다. 이때 v\$sqllock을 조회한 결과가 아래와 같다. 이 락 상태에 대한 해석으로 가장 적절하지 않은 것은?

아래

[v\$sqllock - 렌탈계약 두 행에 엇갈린 TX 락]						
SID	TYPE	ID1	ID2	LMODE	REQUEST	BLOCK
30	TX	327681	14522	6	0	1
30	TX	393225	14870	0	6	0
52	TX	393225	14870	6	0	1
52	TX	327681	14522	0	6	0

* TX 모드 코드: 6 = X(Exclusive)

* BLOCK=1 은 그 자원을 보유해 상대 세션을 막고 있음을 뜻한다.

- ① 세션 30은 자신이 잠근 행의 TX 락(327681·14522)을 X(6)로 보유(BLOCK=1)하면서, 세션 52가 잠근 행의 TX 락(393225·14870)을 X로 REQUEST해 대기한다.
- ② 세션 52도 대칭으로 393225·14870을 X로 보유(BLOCK=1)하고 327681·14522를 REQUEST하므로, 두 세션이 서로의 자원을 기다리는 순환 대기가 형성됐다.
- ③ 오라클은 이 순환 대기를 자동으로 감지해, 두 세션 중 한쪽의 문장(statement)을 롤백하고 ORA-00060 오류를 던져 교착을 해소한다. 이때 롤백되는 것은 문장이지 그 세션의 트랜잭션 전체가 소멸하는 것은 아니다.
- ④ 두 세션 모두 BLOCK=1이지만 이는 한쪽만 상대를 막는 단방향 블로킹이라 교착이 아니며, 오라클은 이런 상황을 감지하지 못해 두 세션이 무한정 서로를 기다린다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ③	3. ②	4. ③	5. ②
6. ②	7. ④	8. ②	9. ①	10. ①
11. ③	12. ①	13. ②	14. ④	15. ④
16. ③	17. ③	18. ①	19. ④	20. ④

문제 1. 정답 ①

버퍼 캐시의 상태 전이는 누가·언제 디스크에 쓰는가가 핵심입니다. 여기서 커밋의 역할과 DBWR의 역할을 갈라야 합니다.

커밋 시점 : LGWR가 Redo 로그를 디스크에 기록(Write Ahead Logging / Fast Commit)

→ 데이터 블록 자체는 이때 디스크에 내려가지 않는다

더티 버퍼 : 변경됐지만 데이터파일 미반영 상태로 캐시에 남는다

DBWR : 이후 비동기로(체크포인트·프리 버퍼 부족 시) 더티 버퍼를 데이터파일에 기록

→ 기록이 끝나야 그 버퍼가 프리 버퍼로 재사용 가능

①은 이 흐름을 정반대로 뒤집었습니다. 더티 버퍼를 디스크에 쓰는 주체는 서버 프로세스가 아니라 DBWR이고, 그 시점도 커밋 순간이 아니라 이후 비동기입니다. 커밋이 보장하는 것은 데이터 블록의 데이터파일 반영이 아니라 Redo 로그의 기록(LGWR)입니다. 만약 커밋마다 변경 블록을 즉시 데이터파일에 내려야 한다면 Fast Commit 구조가 성립하지 않습니다. "커밋 즉시 서버 프로세스가 직접 데이터 블록을 기록한다"는 서술은 커밋의 대상(Redo)과 데이터 블록 기록(DBWR)을 뒤바꾼 것입니다.

②·③·④는 옳습니다. 터치 카운트를 반영한 LRU 관리(②), 캐시 미스 시 프리 버퍼 확보와 부족 시 DBWR 기동(③), 더티·프리 버퍼의 정의와 DBWR 기록 후의 상태 전이(④)가 모두 버퍼 캐시 관리 원리에 부합합니다.

선택지	판정	이유
①	×	더티 버퍼를 데이터파일에 쓰는 주체는 DBWR이고 시점은 커밋 이후 비동기입니다. 커밋이 보장하는 것은 LGWR의 Redo 기록이지 데이터 블록 반영이 아닙니다 — 주체와 대상을 뒤집은 서술입니다
②	○	오라클은 블록 접근 시 터치 카운트를 올리고 이를 반영한 LRU로 재사용 대상을 고릅니다. 자주 쓰는 블록이 오래 남습니다
③	○	캐시 미스 시 LRU에서 프리 버퍼를 확보해 디스크 블록을 적재하고, 프리 버퍼가 부족하면 DBWR를 깨워 더티 버퍼를 내려 공간을 비웁니다
④	○	더티·프리 버퍼의 정의가 맞고, DBWR 기록으로 내용이 디스크와 일치하면 더티 버퍼가 프리 버퍼로 전환됩니다

■ 이 문제의 핵심

1. 커밋 = Redo 기록(LGWR), 데이터 블록 기록 = DBWR(비동기). 커밋 순간에 변경 블록이 데이터파일로 내려가는 것이 아닙니다.
2. 더티 버퍼를 데이터파일에 쓰는 주체는 서버 프로세스가 아니라 DBWR입니다.
3. 오라클은 터치 카운트 기반 LRU로, 자주 참조되는 블록을 캐시에 오래 유지합니다.
4. 더티 → (DBWR 기록) → 프리의 상태 전이를 거쳐야 버퍼가 재사용됩니다.

■ 한 줄 정리 커밋이 즉시 기록하는 것은 Redo 로그(LGWR)이고 데이터 블록은 DBWR가 이후 비동기로 내리므로, 더티 버퍼를 서버 프로세스가 커밋 순간 직접 기록한다는 서술은 주체와 시점을 뒤집은 것입니다.

문제 2. 정답 ③

대량 스캔에는 캐시를 거치는 경로와 캐시를 우회하는 경로가 따로 있고, 대기 이벤트도 갈립니다. 이 경계를 뭉개면 안 됩니다.

버퍼 캐시 경우 Multiblock I/O

경로 : 디스크 → 버퍼 캐시(SGA) → 서버 프로세스

대기 : db file scattered read

전형 : (직렬) Full Table Scan, Index Fast Full Scan

Direct Path Read (캐시 우회)

경로 : 디스크 → 프로세스 PGA (SGA 버퍼 캐시를 거치지 않음)

대기 : direct path read

전형 : 병렬 쿼리 스캔, 대량 세그먼트 스캔

전제 : 대상 세그먼트의 더티 버퍼를 먼저 디스크로 내리는 체크포인트 수행

③은 Direct Path Read의 본질을 정확히 짚었습니다. 읽은 블록을 SGA 버퍼 캐시에 두지 않고 곧바로 PGA로 가져오며, 그래서 대기 이벤트도 **db file scattered read**가 아니라 **direct path read**로 관측됩니다.

나머지는 두 경로의 경계를 흐렸습니다.

①은 Direct Path Read까지 "버퍼 캐시에 적재되고 **scattered read**로 관측된다"고 묶었지만, Direct Path Read는 캐시를 우회하며 이벤트도 **direct path read**입니다.

②는 Direct Path Read가 캐시의 더티 버퍼를 재활용한다고 했으나, 실제로는 그 반대입니다. Direct Path로 읽기 전에 대상 세그먼트의 더티 버퍼를 디스크로 내리는 체크포인트를 먼저 수행한 뒤 디스크에서 직접 읽습니다. 캐시 블록을 활용하는 것이 아니라 비워 내는 것입니다.

④는 병렬 Full Scan의 기본 경로를 반대로 봤습니다. 병렬 슬레이브의 대량 스캔은 캐시 오염을 피하러 Direct Path Read를 기본으로 씁니다.

선택지	판정	이유
①	×	Direct Path Read는 버퍼 캐시를 우회하고 대기 이벤트도 direct path read 입니다. 두 경로를 scattered read ·캐시 적재로 통일한 것은 경계를 뭉개는 서술입니다
②	×	Direct Path Read는 읽기 전에 대상 세그먼트의 더티 버퍼를 디스크로 내린 뒤 직접 읽습니다. 캐시의 더티 버퍼를 재활용해 디스크를 건너뛴다는 것은 방향이 반대입니다
③	○	Direct Path Read는 SGA를 거치지 않고 PGA로 직접 적재하며, 대기 이벤트는 db file scattered read 가 아니라 direct path read 입니다. 경로와 이벤트가 정확합니다
④	×	병렬 Full Scan은 캐시 오염을 피해 Direct Path Read를 기본 경로로 씁니다. Multiblock I/O(캐시 경유)를 기본이라 한 것은 병렬 스캔의 경로를 뒤집은 서술입니다

▪ 이 문제의 핵심

1. Multiblock I/O(캐시 경유) ↔ **db file scattered read**, Direct Path Read(캐시 우회) ↔ **direct path read**. 대기 이벤트가 경로를 가릅니다.

2. Direct Path Read는 블록을 SGA가 아니라 PGA로 직접 가져옵니다 — 캐시 재활용이 아니라 캐시 우회입니다.

3. Direct Path로 읽기 전, 대상 세그먼트의 더티 버퍼를 먼저 디스크로 내리는 체크포인트가 선행됩니다.

4. 병렬 Full Scan의 기본 경로는 Direct Path Read입니다 — 캐시 경유 Multiblock이 아닙니다.

▪ 한 줄 정리 대량 스캔에서 캐시를 우회해 PGA로 직접 읽는 Direct Path Read의 대기 이벤트는 **direct path read**이며, **scattered read**(캐시 경유 Multiblock)와 같은 경로로 묶으면 안 됩니다.

문제 3. 정답 ②

먼저 재귀 Call의 비중을 확인합니다.

전체 DB Call = 비재귀 Total 4 + 재귀 Total 198,000 = 198,004

재귀 Call 비율 = 198,000 / 198,004 ≈ 99.998% ← 수행의 거의 전부가 재귀

재귀가 이렇게 폭증한 원인은 파싱 통계에 드러납니다.

재귀 Parse = 50,000회

라이브러리 캐시 Miss(parse) = 50,000회

하드파싱 비율 = 50,000 / 50,000 = 100% ← 파싱마다 캐시 미스 = 전부 하드파싱

사원번호를 리터럴로 조립했으므로 **WHERE 사원번호 = 10023, = 10024** ... 는 SQL 텍스트가 매번 달라 커서를 공유하지 못하고 5만 번 모두 하드파싱됐습니다. 이 하드파싱이 곧 재귀 SQL(딕셔너리 조회·커서 생성)을 유발한 축입니다. 파싱이 얼마나 무거운지 봅니다.

재귀 Parse CPU = 12.4초 / 재귀 CPU 27.7초 ≈ 44.8% ← 재귀 CPU의 절반이 파싱

재귀 Parse 시간 = 18.6초 > 재귀 Execute 14.1초 ← 실행보다 파싱이 더 오래

재귀 CPU 27.7초 / 전체 CPU 27.713초 ≈ 99.95% ← 비용도 재귀에 집중

파싱이 재귀 CPU의 45%, 재귀 Elapsed에서도 Execute를 웃돕니다. 처방은 바인드 변수로 커서를 공유해 하드파싱 5만 번을 1번으로 줄이는 것입니다. 한편 재귀 Call 수가 큰 것은 파싱·내부 SQL 축의 이야기일 뿐, 결과 10만 행의 정확성이나 데이터량과는 별개의 축입니다. 따라서 ②이 옳은 진단입니다.

선택지	판정	이유
①	×	재귀 Call 198,000은 5만 번의 하드파싱이 유발한 파싱·내부 SQL이지 데이터량 지표가 아닙니다. 대상 사원을 줄여도 리터럴을 쓰는 한 사원당 하드파싱은 그대로여서 파싱 부하 축을 겨냥하지 못합니다
②	○	재귀 Parse 50,000회에 라이브러리 캐시 Miss 50,000회(100%)로 전부 하드파싱이고, 파싱이 재귀 CPU 27.7초의 44.8%(12.4초)를 차지합니다. 바인드 변수로 커서를 공유하면 파싱 부하가 붕괴합니다
③	×	결과 10만 행은 발문에서 정확하다고 명시됐고, 재귀 Call이 많다는 사실과 결과 정확성은 서로 다른 축입니다. 재귀 SQL은 정상 처리의 부산물이니 오류 신호가 아닙니다
④	×	파싱 비용이 크다는 것과 실행계획 품질은 별개의 축입니다. 하드파싱 5만 번은 텍스트가 매번 달라 커서를 공유 못한 탓이지 계획이 나빠서가 아니므로, 힌트로 계획을 고정해도 하드파싱은 사라지지 않습니다

■ 이 문제의 핵심

1. 재귀 Call 비율 = 재귀 Total ÷ 전체 Call. 여기서는 198,000/198,004 ≈ 100%로 수행의 거의 전부가 재귀입니다.
2. 하드파싱 판정은 **Misses in library cache during parse**. 재귀 Parse 50,000회에 Miss 50,000회면 100%가 하드파싱입니다.
3. 리터럴로 조립한 SQL은 텍스트가 매번 달라 커서를 공유하지 못하고, 그 하드파싱이 재귀 SQL을 유발합니다. 처방은 바인드 변수입니다.
4. 재귀 Call 수는 파싱·내부 SQL의 축이지 결과 데이터량·정확성·실행계획 품질의 축이 아닙니다 — 이 둘을 인과로 묶으면 오진입니다.
5. 파싱이 재귀 CPU의 절반(12.4초/27.7초)을 먹었다는 사실이 병목을 '파싱'으로 특정해 줍니다.

■ 한 줄 정리 재귀 Call 198,000의 뿌리는 리터럴이 부른 하드파싱 5만 번(라이브러리 캐시 Miss 100%)이며, 이는 데이터량·결과 정확성·계획 품질과는 별개의 축이므로 처방은 바인드 변수입니다.

문제 4. 정답 ③

Fetch 횟수와 Rows로 배열 크기(ArraySize)를 역산합니다. 마지막 한 번은 "더 없음(no-more-data)" 인출이므로 -1 합니다.

$$\begin{aligned}
 \text{ArraySize} &= \text{총 Rows} \div (\text{Fetch 횟수} - 1) \\
 &= 360,000 \div (3,601 - 1) \\
 &= 360,000 \div 3,600 = 100 \text{ (행/Fetch)}
 \end{aligned}$$

한 번에 100행씩만 담아 오니 36만 행을 나르는 데 3,601번의 인출 왕복이 필요했습니다. 다음은 시간이 어디서 새는지입니다.

$$\begin{aligned}
 \text{스캔 실작업 시간} &= \text{Row Source time} = 4.12\text{초} && \leftarrow \text{DB가 실제로 블록을 읽은 시간} \\
 \text{Fetch Elapsed} &= 28.8\text{초} \\
 \text{간극} &= 28.8 - 4.12 \approx 24.7\text{초} && \leftarrow \text{DB 작업이 아니라 인출 왕복 대기} \\
 \text{CPU} &= 3.2\text{초 (Elapsed의 약 11\%)} && \leftarrow \text{CPU 바운드도 아님}
 \end{aligned}$$

DB가 블록을 읽은 실작업은 4.1초뿐인데 응답은 28.8초입니다. 그 24.7초는 100행마다 한 번씩 클라이언트를 오간 왕복 대기입니다. 물리 읽기는 900블록에 불과해 디스크가 25초를 만들 수 없고, 캐시 적중도 높습니다.

$$\begin{aligned}
 \text{BCHR} &= (180,500 - 900) / 180,500 \times 100 = 99.5\% && \leftarrow \text{논리 읽기의 99.5\%가 캐시 적중} \\
 \text{디스크 비중} &= 900 / 180,500 \approx 0.5\%
 \end{aligned}$$

처방은 스토리지·인덱스·캐시가 아니라 배열 크기 상향입니다. 100→1,000이면 Fetch가 360,000/1,000+1 ≈ 361회로 약 1/10이 되어 왕복 대기가 그만큼 줄어듭니다. 따라서 ③이 옳습니다.

선택지	판정	이유
①	×	물리 읽기는 900블록뿐이라 24초대 대기를 만들 수 없습니다. 지연의 정체는 디스크가 아니라 3,601회 인출 왕복이므로 스토리지 교체는 원인을 잘못 짚은 처방입니다
②	×	cr 180,500은 스캔 실작업으로 4.1초에 끝났고, 병목은 논리 읽기가 아니라 왕복 대기입니다. 인덱스를 만들어도 배열 크기가 100이면 36만 행을 3,601번에 나눠 나르는 왕복은 그대로 남습니다
③	○	360,000/(3,601-1)=100으로 배열 크기가 작아 Fetch가 3,601회 발생했고, Fetch Elapsed 28.8초와 스캔 4.1초의 24.7초 간극이 왕복 대기입니다. 배열 크기를 1,000으로 올리면 Fetch가 약 361회로 줄어 대기가 함께 감소합니다
④	×	BCHR 99.5%와 스캔 4.1초까지는 옳지만, 남은 24.7초를 '손댈 수 없는 네트워크 지연'으로 본 것이 오귀속입니다. 왕복 횟수는 배열 크기로 줄일 수 있어 SQL·설정 차원에서 감축 가능합니다

■ 이 문제의 핵심

1. ArraySize = 총 Rows ÷ (Fetch 횟수 - 1). 여기서는 360,000/(3,601-1) = 100입니다. 마지막 no-more-data 인출 때문에 -1 합니다.
2. Fetch Elapsed와 Row Source time의 간극이 왕복 대기입니다. 28.8초 - 4.12초 ≈ 24.7초가 DB 실작업이 아니라 인출 왕복이었습니니다.
3. 물리 읽기 900·CPU 3.2초·BCHR 99.5%는 모두 디스크·CPU·캐시가 병목이 아님을 말해 줍니다 — 남은 측은 왕복뿐입니다.
4. 처방은 배열 크기 상향입니다. 100→1,000이면 Fetch가 약 1/10(3,601→361회)로 줄어 왕복 대기가 비례해 감소합니다.
5. 인덱스·스토리지·PGA로 눈을 돌리면 Fetch 지연을 엉뚱한 요인에 오귀속하게 됩니다.
 - 한 줄 정리 Fetch 3,601회는 배열 크기 100(=360,000/3,600)의 결과이고, Fetch Elapsed와 스캔 time의 24.7초 간극은 왕복 대기이므로 처방은 디스크·인덱스가 아니라 배열 크기 상향입니다.

문제 5. 정답 ②

두 함수는 어디에서 계획을 읽어 오는가가 다릅니다. 이 출처 경계가 문제의 핵심입니다.

DBMS_XPLAN.DISPLAY

출처 : PLAN_TABLE (EXPLAIN PLAN이 적재한 예상 계획)
 성격 : 문장 미수행, 추정치 기반 예상 계획
 바인드 : 값을 엿보지 못함 → 기본 가정으로 계획 산출

DBMS_XPLAN.DISPLAY_CURSOR

출처 : 라이브러리 캐시 / V\$SQL_PLAN (실제 수행된 커서)
 성격 : 실제 수행한 문장의 실제 실행계획
 바인드 : 첫 수행 시 실제 값을 엿봄(bind peeking) → 그 값에 맞춘 계획

②은 이 경계를 정확히 갈랐습니다. **DISPLAY**는 **EXPLAIN PLAN**이 남긴 예상 계획을, **DISPLAY_CURSOR**는 실제 수행된 커서의 계획을 조회합니다. 그리고 바인드 변수가 있으면 **EXPLAIN PLAN**은 그 값을 엿보지 못한 채 기본 가정으로 계획을 세우므로, 실제 수행 시 bind peeking을 거친 계획과 달라질 수 있습니다. 예상 계획을 신뢰하다 실제와 어긋나는 전형적 지점입니다.

나머지는 두 함수의 출처·성격을 뒤섞었습니다.

①는 **DISPLAY_CURSOR**를 "PLAN_TABLE의 예상 계획을 읽는 함수"로 봤지만, 이 함수는 라이브러리 캐시의 실제 수행 커서를 읽습니다. **DISPLAY**와 서로 바꿔 쓸 수 있는 것이 아닙니다.

③은 **EXPLAIN PLAN**이 문장을 수행하고 바인드 값을 반영한다고 했으나, **EXPLAIN PLAN**은 문장을 수행하지 않고 바인드 값도 엿보지 않습니다. 그래서 예상과 실체가 갈릴 수 있습니다.

④는 **TRACEONLY EXPLAIN**을 실제 커서 조회로 오해했습니다. 이 옵션은 **EXPLAIN PLAN** 방식의 예상 계획을 낼 뿐, 문장을 수행하지 않아 실측 행 수가 담기지 않습니다.

선택지	판정	이유
①	×	DISPLAY_CURSOR 는 PLAN_TABLE이 아니라 라이브러리 캐시의 실제 수행 커서를 읽습니다. DISPLAY 와 바꿔 쓸 수 있는 예상 계획 함수가 아닙니다
②	○	DISPLAY 는 PLAN_TABLE의 예상 계획, DISPLAY_CURSOR 는 실제 수행된 커서의 계획을 조회하며, EXPLAIN PLAN 은 bind peeking을 못 해 실체와 갈릴 수 있습니다. 출처와 원인이 정확합니다
③	×	EXPLAIN PLAN 은 문장을 수행하지도, 바인드 값을 엿보지도 않습니다. 예상 계획과 실제 커서 계획이 갈릴 수 있는 이유가 바로 이것입니다
④	×	TRACEONLY EXPLAIN 은 EXPLAIN PLAN 방식의 예상 계획을 낼 뿐 문장을 수행하지 않아 실측 행 수가 없고, 실제 커서에서 가져온 것도 아닙니다

▪ 이 문제의 핵심

1. **DISPLAY** = PLAN_TABLE의 예상 계획, **DISPLAY_CURSOR** = 라이브러리 캐시의 실제 커서 계획. 출처가 다릅니다.
2. **EXPLAIN PLAN**은 문장을 수행하지 않고 바인드 값을 엿보지 않습니다 — 그래서 예상 계획이 나옵니다.
3. 실제 수행은 첫 파싱 때 바인드 값을 엿보므로(bind peeking), 예상 계획과 다른 계획이 나올 수 있습니다.
4. **TRACEONLY EXPLAIN**은 예상 계획 도구입니다. 실측 행 수가 담긴 실제 계획을 보려면 실제 수행 후 **DISPLAY_CURSOR**를 써야 합니다.

▪ 한 줄 정리 **DISPLAY**는 PLAN_TABLE의 예상 계획을, **DISPLAY_CURSOR**는 실제 수행된 커서의 계획을 조회하며, **EXPLAIN PLAN**은 바인드 값을 엿보지 못해 예상과 실체가 갈릴 수 있습니다.

문제 6. 정답 ②

예금거래_IX는 (예금계좌번호, 거래일자, 거래구분) 순서입니다. 인덱스로 스캔 시작·종료 지점(스캔 범위)을 좁힐 수 있는 조건은, 선두부터 등치(=)로 이어지다가 처음 만난 범위(부등호) 조건까지입니다.

예금계좌번호 = '110-204-778' → 등치, 선두 칼럼 : 액세스 조건 (스캔 시작·종료 결정)
 거래일자 >= ... AND < ... → 범위, 두 번째 칼럼 : 액세스 조건 (스캔 종료 결정)
 거래구분 = '이체' → 범위 조건 뒤의 세 번째 칼럼 : 스캔 범위 못 좁힘 → 인덱스 필터

거래구분은 등치(=) 조건이지만, 자기 앞 칼럼(거래일자)이 범위 조건이라 인덱스 정렬 순서상 연속 구간을 더 자를 수 없습니다. 그래서 스캔 시작·종료 지점은 **예금계좌번호=, 거래일자 범위**까지로 정해지고, 그 구간을 훑는 동안 거래구분='이체'는 인덱스 필터로 각 엔트리를 걸러냅니다.

거래구분이 인덱스 필터로 처리된다는 근거는 두 노드의 Card가 같다는 점입니다.

INDEX RANGE SCAN 예금거래_IX Card 940 ← 거래구분 필터까지 적용해 인덱스에서 나온 건수
 TABLE ACCESS BY INDEX ROWID 예금거래 Card 940 ← 테이블에서 추가로 줄지 않음

거래구분이 인덱스에 있으므로 필터가 인덱스 단계에서 끝나 테이블 액세스로 넘어갈 때 건수가 940으로 그대로입니다(테이블 필터였다면 여기서 더 줄었을 것입니다). 이 구조를 정확히 서술한 것이 ②입니다. 세 칼럼 모두 액세스 조건이라는 ①, 예금계좌번호 하나만 액세스 조건이라는 ③, 거래구분을 테이블 필터로 본 ④는 모두 이 스캔 범위·Card 흐름과 어긋납니다.

선택지	판정	이유
①	×	거래구분은 범위 조건(거래일자) 뒤 칼럼이라 스캔 시작·종료 지점을 정하는 데 못 쓰입니다. 액세스 조건은 예금계좌번호·거래일자까지이며, 세 번째 칼럼은 스캔 범위를 좁히지 못합니다
②	○	예금계좌번호(=)·거래일자(범위)가 스캔 범위를 정하는 액세스 조건, 거래구분은 범위 뒤 칼럼이라 인덱스 필터 — INDEX RANGE SCAN과 TABLE ACCESS의 Card가 940으로 같은 흐름과 일치합니다
③	×	거래일자 범위도 엄연히 스캔 종료 지점을 정하는 액세스 조건입니다. 액세스 조건을 예금계좌번호 하나로 좁힌 것은 부분을 전체로 끼운 진술입니다
④	×	거래구분은 예금거래_IX 의 세 번째 구성 칼럼이라 인덱스 필터로 처리됩니다. 테이블 필터였다면 Card가 940보다 줄었어야 하는데 두 노드 모두 940으로 같습니다

■ 이 문제의 핵심

- 스캔 범위(액세스 조건)는 선두부터 첫 범위 조건까지. 여기서는 예금계좌번호(=)·거래일자(범위)가 INDEX RANGE SCAN의 시작·종료 지점을 정합니다.
- 범위 조건 뒤 칼럼은 인덱스 필터. 거래구분은 등치라도 앞 칼럼(거래일자)이 범위여서 스캔 범위를 못 좁히고, 스캔 구간 안에서 필터로만 걸러집니다.
- 인덱스 필터 vs 테이블 필터는 Card로 구분. 거래구분이 인덱스에 있어 INDEX RANGE SCAN·TABLE ACCESS의 Card가 940으로 같습니다 — 테이블 단계에서 줄지 않았습니다.
- 인덱스에 있다고 다 액세스 조건은 아닙니다. 스캔 범위를 좁히는 것(액세스)과 스캔 구간 안을 거르는 것(필터)은 다릅니다.

■ 한 줄 정리 예금계좌번호 등치와 거래일자 범위까지가 스캔 범위를 정하는 액세스 조건이고, 범위 조건 뒤의 거래구분은 스캔 범위를 못 좁혀 인덱스 필터로 처리되므로 INDEX RANGE SCAN과 TABLE ACCESS의 Card가 940으로 같습니다.

문제 7. 정답 ④

트레이스의 세 숫자를 나란히 놓습니다.

500,000 ← 인덱스에서 얻은 ROWID 수 (INDEX RANGE SCAN 의 Rows)
 121,600 ← 테이블 액세스 누적 블록 I/O (TABLE ACCESS 의 cr)
 1,600 ← 인덱스 스캔에 든 블록 (INDEX RANGE SCAN 의 cr)

부모(TABLE ACCESS) cr에는 자식(INDEX) cr이 누적돼 있으므로, 순수 테이블 랜덤 블록을 분리합니다.

테이블 랜덤 블록 = 121,600 - 1,600 = 120,000
 랜덤 블록 비중 = 120,000 / 121,600 ≈ 98.7% ← cr의 거의 전부가 테이블 랜덤

클러스터링 팩터(CF)의 밀도를 봅니다. 이론상 최소 밀도는 테이블 블록/행 = 80,000/8,000,000 ≈ 0.01입니다.

CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수 = 120,000 / 500,000 = 0.24 (ROWID당 블록)
 CF 추정 ≈ 0.24 × 8,000,000 = 1,920,000
 → 테이블 블록 80,000의 약 24배 (최소 밀도 0.01의 24배)
 → 나쁜지만 최악(1.0)은 아닌 중간값

읽은 행 비율은 500,000/8,000,000 ≈ 6.25%로 흔히 '인덱스가 유리'로 보이는 범위지만, 나쁜 CF 탓에 단일블록 랜덤 읽기가 12만 블록에 이릅니다. 이제 세 경로를 손익분기점으로 비교합니다.

현재 경로(인덱스+테이블) = 120,000(랜덤) + 1,600(인덱스) = 121,600 블록
 Full Scan = 80,000 블록 (multiblock)
 커버링 인덱스 스캔 ≈ 인덱스만 읽음 → 컬럼 2개 추가로 리프가 넓어져도 수천 블록대

Elapsed 31.5초 대비 CPU 4.9초의 약 26.6초 간극은 물리 읽기(Disk 40,000) 대기이며, 그 뿌리는 테이블 랜덤 액세스입니다. SELECT가 **참고이동일자·이동수량** 두 컬럼뿐이므로 이 둘을 인덱스에 얹은 커버링 인덱스면 **TABLE ACCESS BY INDEX ROWID**가 사라져 랜덤 블록 12만이 통째로 없어집니다. cr은 인덱스만 읽는 수준으로 떨어져 Full Scan 80,000보다도 적습니다. 병목은 인덱스가 아니라 테이블 랜덤 액세스이고, 최적 처방은 테이블 액세스 제거(커버링)이므로 ④가 옳습니다.

선택지	판정	이유
①	×	(121,600-40,000)/121,600 ≈ 67%로 BCHR 계산은 맞지만, 낭비의 정체는 캐시가 아니라 논리 읽기 121,600 자체(나쁜 CF)입니다. 캐시를 키워도 12만 블록 랜덤 액세스는 남습니다
②	×	랜덤 121,600 > Full Scan 80,000까지는 맞지만, 두 컬럼만 읽으므로 커버링 인덱스가 테이블 액세스를 없애 cr을 약 1,600으로 낮춰 Full Scan 80,000보다 더 유리합니다. Full Scan이 유일한 해법이라는 단정은 틀립니다
③	×	ROWID당 0.24는 최소 밀도(0.01)의 24배로 오히려 나쁜 쪽입니다. 31초는 리프 블록 분할(인덱스 cr은 1,600뿐)이 아니라 테이블 랜덤 액세스의 물리 I/O 대기입니다
④	○	테이블 랜덤 블록 120,000 / ROWID 500,000 = 0.24로 CF는 테이블 블록의 약 24배(나쁨)이고 cr의 98.7%가 랜덤입니다. 두 컬럼만 읽으므로 커버링으로 테이블 액세스를 없애면 cr이 약 1,600으로 떨어져 Full Scan 80,000보다도 유리합니다

■ 이 문제의 핵심

- 테이블 랜덤 블록 = TABLE ACCESS cr - INDEX cr = 121,600 - 1,600 = 120,000. 부모 cr에는 자식 인덱스 cr이 누적돼 있으니 반드시 빼서 분리합니다.
- CF 밀도 ≈ 테이블 랜덤 블록 ÷ ROWID 수. 0.24는 최소 밀도(약 0.01)의 24배로, 최악(1.0)은 아니어도 분명히 나쁜 중간값입니다.
- cr의 98.7%가 테이블 랜덤이므로 병목은 인덱스가 아니라 테이블 랜덤 액세스입니다.
- 손익분기점 비교: 현재 121,600 > Full Scan 80,000 > 커버링 약 1,600대. 읽는 컬럼이 적으면 Full Scan보다 커버링이 더 유리합니다.
- 높은 논리 읽기 자체가 낭비라면 캐시 확대로는 해결되지 않습니다 — 테이블 액세스를 제거(커버링)해야 랜덤 블록 12만이 사라집니다.

■ 한 줄 정리 ROWID 50만에 테이블 랜덤 12만이면 CF는 테이블 블록의 약 24배로 나쁘고, SELECT가 두 컬럼뿐이라 커버링 인덱스로 테이블 액세스를 없애는 것이 Full Scan보다도 유리한 처방입니다.

문제 8. 정답 ②

두 스캔은 읽는 순서가 다르고, 그 차이가 정렬 보장 여부를 가릅니다.

Index Full Scan

읽는 순서 : 리프 블록의 논리적 연결 순서 (인덱스 키 순서)
 I/O : 한 블록씩 Single Block I/O
 정렬 : 인덱스 키 순서 보장 → ORDER BY 소트 생략 가능

Index Fast Full Scan

읽는 순서 : 세그먼트의 물리적 저장 순서 (연결 순서 무시)
 I/O : 인접 블록을 묶는 Multiblock I/O, 병렬 가능
 정렬 : 키 순서 보장 안 됨 → ORDER BY 소트 생략 불가

②는 이 관계를 정반대로 진술했습니다. 인덱스 키 순서로 정렬된 결과를 주는 것은 Index Full Scan입니다. Index Fast Full Scan은 리프 블록을 논리적 연결 순서가 아니라 물리적 저장 순서로 훑기 때문에 결과가 키 순서로 나오지 않으며, 그래서 **ORDER BY**를 소트 없이 대체할 수 없습니다. 정렬 보장을 두 스캔 사이에서 맞바꿔 붙인 서술입니다.

①·④·③은 옳습니다. Index Full Scan의 Single Block·키 순서 보장(①), Index Fast Full Scan의 Multiblock·병렬 스캔(④), 그리고 필요한 컬럼이 인덱스에 다 있을 때의 인덱스 전용 스캔(테이블 액세스 생략, ③)이 두 스캔의 성질에 부합합니다.

선택지	판정	이유
①	○	Index Full Scan은 리프 블록을 논리적 연결 순서로 Single Block I/O로 훑어 키 순서 결과를 주므로 ORDER BY 소트를 생략할 수 있습니다
②	×	방향이 뒤집혔습니다. 키 순서 정렬을 보장하는 것은 Index Full Scan이고, Fast Full Scan은 물리적 순서로 읽어 정렬을 보장하지 못해 ORDER BY 를 대체할 수 없습니다
③	○	필요한 컬럼이 인덱스에 다 있으면 어느 방식이든 테이블 액세스를 생략하고 인덱스만 읽어 결과를 낼 수 있습니다(인덱스 전용 스캔)
④	○	Index Fast Full Scan은 세그먼트를 물리적 순서대로 Multiblock I/O로 읽어 대량 스캔에 유리하고 병렬 수행이 가능합니다

■ 이 문제의 핵심

1. Index Full Scan = 논리적 연결 순서·Single Block I/O·키 순서 보장. **ORDER BY** 소트를 생략할 수 있습니다.
2. Index Fast Full Scan = 물리적 저장 순서·Multiblock I/O·병렬 가능·정렬 미보장. 대량 스캔엔 빠르지만 순서를 못 줍니다.
3. 정렬이 필요하다면 Index Full Scan, 순서가 필요 없는 대량 처리면 Index Fast Full Scan — 정렬 보장 여부가 갈림길입니다.
4. 필요한 컬럼이 인덱스에 다 있으면 테이블 액세스를 생략하는 인덱스 전용 스캔이 되고, 이는 어느 방식에서든 가능합니다.

■ 한 줄 정리 인덱스 키 순서로 정렬된 결과를 주어 **ORDER BY**를 대체하는 것은 Index Full Scan이며, 물리적 순서로 읽는 Index Fast Full Scan은 정렬을 보장하지 못합니다.

문제 9. 정답 ①

인덱스가 키를 오름차순으로 저장하는 것은 맞지만, 그렇다고 내림차순 정렬을 인덱스로 대체할 수 없는 것은 아닙니다. 여기서 경계를 잘못 그으면 안 됩니다.

인덱스 저장 : 관측소코드 오름차순으로 정렬 보관
 ORDER BY ASC : 인덱스를 앞에서부터(정방향) 읽음 → 소트 생략
 ORDER BY DESC : 인덱스를 뒤에서부터(역방향) 읽음 → 소트 생략(내림차순 인덱스 스캔)

①은 "오름차순 저장이니 DESC는 인덱스로 못 푼다"고 경계를 좁혔지만, 옵티마이저는 인덱스를 역방향으로 읽는 내림차순 인덱스 스캔으로 **ORDER BY ... DESC**도 소트 없이 처리할 수 있습니다. 인덱스는 양방향 연결 리스트로 이어져 있어 앞에서든 뒤에서든 순서대로 훑을 수 있기 때문입니다. "DESC는 저장 순서와 반대라 별도 소트가 필요하다"는 것은 오름차순 저장이라는 사실을 정렬 방향 제약으로 잘못 확장한 경계 오해입니다.

②·③·④는 옳습니다. 선두 컬럼을 등치로 고정한 구간 안에서 후행 정렬이 살아 소트를 생략할 수 있고(②), 정렬 컬럼의 순서·방향이 인덱스와 일치하면 인덱스 읽기만으로 정렬이 해결되며(③), 가운데 컬럼 조건이 빠지면 그 뒤 컬럼(항목코드)은 스캔 범위를 못 좁히고 필터로만 걸러집니다(④).

선택지	판정	이유
①	×	오름차순 저장이라도 인덱스를 역방향으로 읽는 내림차순 인덱스 스캔으로 ORDER BY ... DESC 의 소트를 생략할 수 있습니다. DESC는 인덱스로 못 푼다고 본 것은 경계를 좁힌 오해입니다
②	○	선두 컬럼을 등치로 한 점에 고정하면 그 구간 안에서 후행 관측일시 순서가 유지되므로 ORDER BY 관측일시 의 소트를 생략할 수 있습니다
③	○	정렬 컬럼의 순서와 방향이 인덱스 구성과 일치하면 인덱스를 순서대로 읽는 것만으로 정렬이 해결되어 소트 연산이 생략됩니다
④	○	가운데 컬럼(관측일시) 조건이 없으면 인덱스 정렬이 그 지점에서 흐트러져, 뒤의 항목코드는 스캔 범위를 못 좁히고 인덱스 필터로만 처리됩니다

■ 이 문제의 핵심

1. 인덱스는 오름차순으로 저장되지만, 역방향(내림차순) 인덱스 스캔으로 **ORDER BY ... DESC**도 소트 없이 처리할 수 있습니다.
2. 선두 컬럼을 등치로 한 점에 고정하면 그 구간 안에서 후행 컬럼 정렬이 살아 **ORDER BY**를 대체합니다.
3. 정렬 컬럼의 순서와 방향이 인덱스 구성과 맞아야 인덱스 읽기만으로 소트를 생략합니다.
4. 결합 인덱스에서 가운데 컬럼 조건이 빠지면 그 뒤 컬럼은 스캔 범위를 못 좁히고 인덱스 필터로만 걸러집니다.

- 한 줄 정리 인덱스가 오름차순으로 저장돼도 역방향 스캔으로 **ORDER BY ... DESC**의 소트를 생략할 수 있으므로, DESC는 인덱스로 정렬을 대체할 수 없다는 서술이 경계를 좁힌 오해입니다.

문제 10. 정답 ①

Id 2의 조인 연산 이름을 그대로 읽습니다.

Id 2 HASH JOIN SEMI ← EXISTS가 세미 조인으로 변환된 노드
 Id 3 TABLE ACCESS FULL 가입자 (probe 대상: 각 가입자 1건)
 Id 4~5 적립내역 (INDEX RANGE SCAN 적립일자 범위) ← 매칭 확인용

EXISTS는 "서브쿼리에 행이 하나라도 있으면 참"인 조건입니다. 옵티마이저는 이를 세미 조인(SEMI)으로 변환합니다. 세미 조인의 핵심은 첫 매칭에서 멈춘다는 것입니다. 가입자 한 건에 대해 적립내역에서 가입자ID가 같고 적립일자 조건을 만족하는 행을 하나 찾는 순간 그 가입자를 결과에 넣고 나머지 매칭은 보지 않습니다. 그 결과:

적립내역에 같은 가입자ID가 100건 있어도 가입자는 1건만 출력됩니다(중복 증식 없음).

일반 조인(INNER JOIN)이라면 100건만큼 가입자가 늘어나지만, 세미 조인은 그렇지 않습니다.

Id 2가 **HASH JOIN SEMI**이므로 상관 조건 **가입자ID = 가입자ID**(등치)를 해시로 매칭합니다. 만약 상관 조건이 비등치(Non-EQUI, 예: 범위 비교)였다면 해시로 매칭할 수 없어 **NESTED LOOPS SEMI**로 풀렸을 것입니다. 여기서는 등치라 **HASH JOIN SEMI**가 성립합니다. 이 구조를 정확히 서술한 것이 ①입니다.

세미 조인(SEMI) : 첫 매칭에서 멈춤 → 왼쪽(가입자) 행 중복 증식 없음 (① 서술)
 안티 조인(ANTI) : 매칭이 없는 행만 통과 → NOT EXISTS 변환 (이 플랜 아님)

② 세미=중복제거로 오해, ③ 세미를 안티로 오해, ④ 세미를 일반 조인으로 오해 — 모두 노드 이름(**HASH JOIN SEMI**)과 세미 조인의 첫-매칭 특성에 어긋납니다.

선택지	판정	이유
①	○	Id 2가 HASH JOIN SEMI 이고, 세미 조인은 가입자 한 건당 첫 매칭에서 멈춰 가입자를 중복 증식 없이 통과시킵니다. EXISTS→세미 변환의 정확한 서술입니다
②	×	세미 조인은 중복 제거(DISTINCT) 연산이 아닙니다. '세미=중복제거'라는 반사적 연상인데, 세미는 왼쪽 행의 중복 증식을 막을 뿐 적립내역을 먼저 유일화하지 않습니다
③	×	노드는 HASH JOIN SEMI 이지 HASH JOIN ANTI 가 아닙니다. 안티(NOT EXISTS)는 매칭 없는 행만 반환하지만, 이 SQL은 EXISTS 라 매칭 있는 가입자를 반환합니다
④	×	세미 조인은 첫 매칭에서 멈추므로 가입자가 매칭 건수만큼 늘지 않습니다. 행이 증식하는 것은 일반 조인이고, SEMI 노드는 그 반대입니다

▪ 이 문제의 핵심

1. EXISTS는 세미 조인으로 변환됩니다. 플랜에 **HASH JOIN SEMI**(또는 **NESTED LOOPS SEMI**)로 나타납니다.
2. 세미 조인은 첫 매칭에서 멈춥니다. 왼쪽(가입자) 한 건당 매칭을 하나만 확인하므로 행이 중복 증식되지 않습니다.
3. 세미 ≠ 중복제거, 세미 ≠ 안티. 세미는 DISTINCT가 아니고, 안티(**NOT EXISTS**)는 매칭 없는 행만 통과시키는 반대 개념입니다.
4. 등치면 HASH JOIN SEMI, 비등치면 NESTED LOOPS SEMI. 상관 조건이 등치(가입자ID=가입자ID)라 해시 세미가 성립합니다.

▪ 한 줄 정리 Id 2 **HASH JOIN SEMI**는 EXISTS가 변환된 세미 조인으로, 가입자 한 건당 첫 매칭에서 멈춰 중복 증식 없이 통과시키므로, 세미를 중복제거(②)·안티(③)·일반 조인(④)으로 본 진술은 모두 틀립니다.

문제 11. 정답 ③

해시 조인의 비용을 가르는 축은 Build Input으로 지은 해시 테이블이 Hash Area(PGA work area)에 통째로 올라가느냐입니다.

in-memory : 해시 테이블이 Hash Area에 다 들어감 → 한 번에 Probe
 one-pass : 못 들어감 → 파티션으로 나눠 Temp에 내렸다가 각 파티션을 1회 재처리
 multi-pass : 파티션이 여전히 커 재분할 → Temp를 여러 번 오가며 처리(가장 느림)

③은 이 파티셔닝 기반 저하 처리를 정확히 진술합니다. Build Input이 Hash Area를 넘으면 옵티마이저는 두 입력을 같은 해시 기준으로 파티션 분할하여 짝이 맞는 파티션끼리만 조인하고, 메모리에 못 담는 파티션은 Temp에 기록했다가 다시 읽습니다. 재분할 없이 끝나면 one-pass, 파티션이 커서 다시 쪼개면 multi-pass입니다. 이 상세는 "작은 쪽을 Build로" 같은 기본 정리보다 한 단계 더 들어간 내용이라 낯설어 보이지만, 해시 조인의 실제 동작 그대로입니다.

나머지는 표준 정리에서 한 요소씩 어긋난 진술입니다.

- ①: 양쪽을 정렬해 맞대는 것은 소트 머지 조인입니다. 해시 조인은 정렬이 아니라 해시 함수로 버킷을 잡습니다.
- ②: Probe Input은 Hash Area에 통째로 올리지 않고 한 행씩 읽어(streaming) 해시 테이블을 조회합니다. 양쪽을 모두 메모리에 올리는 구조가 아닙니다.
- ④: 해시 조인은 해시 버킷에 같은 값을 모으는 방식이라 등치(=) 조인에서만 성립합니다. 부등호·범위 조인에는 쓸 수 없습니다.

선택지	판정	이유
①	×	두 입력을 정렬해 맞대는 것은 소트 머지 조인입니다. 해시 조인은 정렬이 아니라 해시 함수로 버킷을 잡아 매칭합니다
②	×	Probe Input은 한 행씩 읽어 해시 테이블을 조회할 뿐, Hash Area에 통째로 올리지 않습니다. 양쪽을 함께 메모리에 적재하지 않습니다
③	○	Build Input이 Hash Area를 넘으면 두 입력을 같은 기준으로 파티션 분할해 Temp를 거쳐 파티션 단위로 조인하며, one-pass·multi-pass로 수행됩니다
④	×	해시 조인은 같은 값을 같은 버킷에 모으므로 등치(=) 조인에서만 성립합니다. 부등호·범위 조인에는 쓰지 못합니다

▪ 이 문제의 핵심

1. Hash Area에 담기면 in-memory, 넘치면 파티셔닝. Build Input이 work area를 넘으면 두 입력을 같은 기준으로 나눠 Temp를 씁니다.
 2. one-pass vs multi-pass. 파티션을 1회 재처리로 끝내면 one-pass, 재분할이 겹치면 multi-pass로 더 느려집니다.
 3. Probe는 streaming. Probe Input은 한 행씩 읽어 해시 테이블을 조회할 뿐, 통째로 메모리에 올리지 않습니다.
 4. 등치 전용. 해시 버킷은 같은 값을 모으는 구조라 부등호·범위 조인에는 쓰지 못합니다 — 정렬해 맞대는 것은 소트 머지 조인입니다.
- 한 줄 정리 해시 조인은 Build 해시 테이블이 Hash Area를 넘으면 두 입력을 같은 기준으로 파티션 분할해 Temp를 거쳐 one-pass·multi-pass로 처리하며, 정렬 병합·Probe 전체 적재·범위 조인 성립은 해시 조인의 표준 원리 밖 서술입니다.

문제 12. 정답 ①

옵티마이저 모드는 서로 독립된 두 목표 중 무엇을 우선할지를 정합니다.

ALL_ROWS : 처리량(throughput) 최적 - 결과 전체를 끝까지 읽는 총비용 최소, 전체범위 최적화
 FIRST_ROWS(n) : 응답 시간(response time) 최적 - 앞쪽 n건을 빨리 반환, 부분범위 처리

이 둘은 다른 축입니다. FIRST_ROWS(n)는 첫 화면을 일찍 띄우려고, 총비용이 조금 더 들더라도 첫 행을 빨리 내보내는 경로(예: 정렬을 피하려 인덱스를 타는 계획)를 고릅니다. 그 결과 앞쪽 응답은 빨라지지만, 결과를 끝까지 다 읽는 총 수행 시간은 오히려 ALL_ROWS보다 늘어날 수 있습니다. 인덱스를 타느라 늘어나는 랜덤 액세스가 전체 건 처리에서는 손해가 되기 때문입니다.

- ①는 "첫 행이 빠르다"(응답 시간)에서 "전체도 빠르다"(처리량)를 곧바로 끌어냈습니다. 두 축을 인과로 묶은 별개 축 혼동이며, 오히려 전체 처리는 ALL_ROWS가 유리한 경우가 많습니다. 그래서 ①가 부적절한 진술입니다.
- ④·②·③은 옳습니다. ALL_ROWS의 처리량·전체범위 최적화(④), FIRST_ROWS(n)의 응답 시간·부분범위 처리와 정렬 회피(②), 규칙 기반 RBO의 폐기와 CBO 기본화(③)가 모두 표준 설명입니다.

선택지	판정	이유
①	×	응답 시간 최적(첫 행 빠름)과 처리량 최적(전체 빠름)은 별개 축입니다. FIRST_ROWS(n)는 총 수행 시간이 오히려 ALL_ROWS보다 늘 수 있어, 둘을 인과로 묶은 것은 축 혼동입니다
②	○	FIRST_ROWS(n)는 앞쪽 n건의 응답 시간을 목표로, 첫 행을 일찍 내보내려 정렬을 인덱스로 대체하는 계획을 선호할 수 있습니다
③	○	RBO는 통계와 무관하게 규칙 순위로 계획을 정하던 옛 방식이며, 지금은 사실상 폐기되고 CBO가 기본입니다
④	○	ALL_ROWS는 결과 전체를 읽는 총비용을 최소화하는 처리량 중심 모드로 전체범위 최적화를 지향합니다

▪ 이 문제의 핵심

1. ALL_ROWS = 처리량 최적(전체범위). 결과를 끝까지 읽는 총비용을 최소화합니다.
2. FIRST_ROWS(n) = 응답 시간 최적(부분범위). 앞쪽 n건을 빨리 내보내려 정렬 회피·인덱스 경로를 선호합니다.
3. 두 목표는 독립 축. 첫 행이 빠르다고 전체가 빠른 것은 아니며, 전체 처리는 ALL_ROWS가 유리한 경우가 많습니다.
4. RBO는 폐기, CBO가 기본. 규칙 순위로 계획을 정하던 옛 방식은 지금 쓰지 않습니다.

- 한 줄 정리 ALL_ROWS는 처리량, FIRST_ROWS(n)는 응답 시간이라는 별개 축을 최적화하므로, 첫 행이 빠르니 전체 수행 시간도 함께 짧아진다는 진술은 두 축을 인과로 묶은 혼동입니다.

문제 13. 정답 ②

쿼리 변환은 이름에서 오는 인상이 아니라 무엇을 어떻게 바꾸는지로 구분해야 합니다.

조건절 이행(Transitive Predicate Generation)은 등치 관계의 연쇄에서 새 조건을 유도하는 변환입니다. **A.key = B.key**이고 **B.key = 10**이면 **A.key = 10**이 논리적으로 성립하므로, 옵티마이저는 이 조건을 만들어 A에 직접 걸어 줍니다. 그 결과 원래는 조인 후에야 좁혀지던 A를 A의 인덱스로 미리 걸러 액세스할 수 있습니다. ②이 이 원리를 정확히 진술합니다. 나머지는 키워드에서 오는 반사적 연상을 노린 함정입니다.

- ①: "뷰 안으로 민다"에서 "뷰가 따로 논다"를 연상하기 쉽지만, 조건절 Pushing은 뷰가 처리할 행을 줄이는 변환이지 병합 여부를 결정하는 변환이 아닙니다. 오히려 병합되지 못한 뷰에 조건을 밀어 넣어 조기 필터링하는 것이 목적이며, 조건이 들어갔다고 블록 분리가 강제되지 않습니다.
- ③: "새 조건 유도 = 옛 조건 대체"로 연상하지만, 조건절 이행은 조인 조건에 새 조건을 더할 뿐 원래의 조인 조건을 지우지 않습니다. **A.key = B.key**가 사라지면 두 테이블을 잇는 연결이 끊겨 결과가 달라집니다.
- ④: 둘 다 "조건을 다룬다"는 인상으로 한 기법처럼 묶었지만, Pushing은 있는 조건을 안으로 미는 것이고 이행은 없던 조건을 유도해 만드는 것으로 하는 일이 다른 독립 변환입니다.

선택지	판정	이유
①	×	Pushing은 뷰가 처리할 행을 줄이는 변환이지 병합 여부를 정하지 않습니다. 조건을 밀어 넣었다고 뷰가 독립 블록으로 분리되지 않습니다
②	○	A.key = B.key 와 B.key = 10 에서 A.key = 10 을 유도해 A를 인덱스 액세스 조건으로 걸 수 있게 하는 것이 조건절 이행입니다
③	×	조건절 이행은 새 조건을 더할 뿐 원래 조인 조건을 제거하지 않습니다. 조인 조건이 사라지면 테이블 간 연결이 끊겨 결과가 달라집니다
④	×	Pushing은 있는 조건을 안으로 밀고 이행은 없던 조건을 유도해 만듭니다. 하는 일이 다른 독립 변환을 한 이름처럼 묶은 반사적 연상 오류입니다

▪ 이 문제의 핵심

1. 조건절 이행 = 새 조건 유도. **A=B ∧ B=10 → A=10**을 만들어 A를 인덱스로 미리 거릅니다.
2. 유도는 더하기지 대체가 아니다. 새 조건이 생겨도 원래 조인 조건은 남습니다 — 지우면 조인이 끊깁니다.
3. Pushing = 있는 조건을 안으로 밀기. 뷰가 처리할 행을 줄일 뿐, 병합 여부나 블록 분리를 결정하지 않습니다.
4. Pushing과 이행은 독립 변환. "둘 다 조건을 다루니 같은 기법"은 키워드에 이끌린 반사적 연상입니다.

- 한 줄 정리 조건절 이행은 등치 연쇄에서 새 조건을 유도해 인덱스 액세스를 열어 주는 변환이며, Pushing과 한 기법으로 묶거나 유도를 조건 대체·블록 분리로 읽는 것은 키워드에 이끌린 반사적 연상 오류입니다.

문제 14. 정답 ④

v\$sql 요약이 그대로 답입니다. 논리적으로 같은 구조의 방식 A 조회가 상담유형 값에 따라 SQL_ID 세 개(a7m2..., f3n8..., k9w4...)로 갈렸고, 각 행이 **EXECUTIONS 1 / LOADS 1**입니다. LOADS 1은 커서를 라이브러리 캐시에 새로 적재(하드파싱)했다는 뜻입니다. 방식 B는 :1로 텍스트가 고정되어 SQL_ID 하나에 EXECUTIONS 3만 붙었습니다.

④는 "오라클 기본 설정에서 옵티마이저가 리터럴을 바인드 변수로 자동 치환해 하나의 커서로 공유한다"는 정규화 가정의 오류입니다. 기본 커서 공유 모드는 **CURSOR_SHARING=EXACT**로, 리터럴을 자동 치환하지 않고 SQL 텍스트를 문자 단위로 비교합니다. 그래서 'INBOUND'·'OUTBOUND'·'VOC'처럼 리터럴만 달라도 텍스트가 달라 각각 별도 커서로 하드파싱됩니다. 만약 ④처럼 자동 치환이 일어난다면 방식 A의 SQL_ID도 하나여야 하는데, 요약표에는 셋입니다. 리터럴을 값과 무관하게 하나로 공유하려면 애초에 방식 B처럼 바인드 변수를 쓰거나 **CURSOR_SHARING=FORCE**를 별도로 지정해야 합니다 — 기본값이 그렇게 해 주지 않습니다. 그래서 부적절한 진술은 ④입니다.

①③②는 모두 "텍스트가 다르면 커서를 공유하지 못한다 / 바인드가 파싱·주입 위험을 함께 줄인다"는 실제 동작을 옳게 서술합니다.

선택지	판정	이유
①	○	방식 A는 값마다 텍스트가 달라 SQL_ID가 셋(a7m2·f3n8·k9w4)으로 갈리고 각 행 LOADS 1로 하드파싱됩니다. 요약표 그대로입니다
②	○	문자열 결합은 입력이 SQL 구문으로 해석될 수 있어 Injection에 노출되고, 값이 다양할수록 하드파싱이 누적돼 라이브러리 캐시 부하가 커집니다
③	○	방식 B는 :1 바인드로 텍스트가 고정되어 SQL_ID 하나(p5c8...)에 EXECUTIONS 3만 붙고, 실행 시점 결합이라 주입에 상대적으로 안전합니다
④	×	기본(EXACT) 모드는 리터럴을 바인드로 자동 치환하지 않습니다. 치환된다면 방식 A의 SQL_ID가 하나여야 하는데 요약표엔 셋입니다

▪ 이 문제의 핵심

- 동적 SQL의 문자열 연결은 값마다 텍스트가 달라져 값의 종류만큼 하드파싱됩니다. SQL_ID 개수와 LOADS가 그 증거입니다.
- 기본 커서 공유(EXACT)는 리터럴을 자동 치환하지 않습니다. "옵티마이저가 알아서 바인드로 묶어 준다"는 것은 정규화 가정의 착각입니다.
- 바인드 변수는 텍스트를 고정해 커서를 공유(EXECUTIONS만 증가)하고, 값이 실행 시점에 결합되어 SQL Injection도 줄입니다.
- 리터럴을 값과 무관하게 공유하려면 바인드 변수를 쓰거나 **CURSOR_SHARING=FORCE**를 별도 지정해야 합니다 — 기본값이 대신해 주지 않습니다.

▪ 한 줄 정리 문자열 연결 동적 SQL은 값마다 다른 SQL_ID로 하드파싱되며, 기본 모드는 리터럴을 자동으로 바인드 치환해 공유해 주지 않으므로 "구조가 같으니 하나로 공유된다"는 것은 착각입니다.

문제 15. 정답 ④

병렬 DML(Parallel DML)로 세그먼트를 수정한 세션은 그 트랜잭션 안에서 같은 객체를 다시 읽거나 고칠 수 없습니다. 커밋(또는 롤백)으로 트랜잭션을 끝내야 비로소 재접근이 허용됩니다. 이 규칙을 어기고 커밋 전에 같은 객체를 조회하면 오라클은 **ORA-12838: cannot read/modify an object after modifying it in parallel** 오류를 던집니다.

[병렬 DML 후 같은 트랜잭션에서의 재접근]

INSERT /*+ APPEND PARALLEL */ ... 수질 측정 → 병렬 DML로 세그먼트 수정
(COMMIT 안 함)
SELECT COUNT(*) FROM 수질 측정 → ORA-12838 (재접근 불가)

허용되는 순서 : ... INSERT ... → COMMIT → SELECT COUNT(*) (그제야 조회 가능)

④은 이 제약을 정반대로 뒤집었습니다. "커밋 없이 같은 트랜잭션에서 곧바로 조회해 확인할 수 있다"고 했지만, 실제로는 커밋 전 재접근이 막혀 오류가 납니다. 방금 넣은 행 수를 확인하려면 먼저 COMMIT 한 뒤 조회해야 합니다. 그래서 부적절한 진술은 ④입니다.

①②③는 병렬 DML의 활성화 전제(①), Direct Path의 HWM 위 기록·PX 서버 병합(②), NOLOGGING+Direct Path의 redo 최소화와 복구 리스크(③)를 모두 옳게 서술합니다.

선택지	판정	이유
①	○	병렬 DML은 세션에서 ENABLE PARALLEL DML 로 활성화해야 INSERT가 병렬로 동작합니다. 활성화 없이는 힌트가 있어도 DML이 직렬로 처리될 수 있습니다
②	○	APPEND는 Direct Path로 HWM 위 새 블록에 기록하고, 병렬 INSERT는 각 PX 서버가 임시 세그먼트에 적재 후 HWM을 올려 병합합니다
③	○	NOLOGGING+Direct Path는 데이터 redo를 최소화해 적재는 빠르나 미디어 복구가 불가능해질 수 있어, 적재 후 백업이 권장됩니다
④	×	병렬 DML 후 커밋 전에는 같은 트랜잭션에서 그 객체를 재조회할 수 없습니다(ORA-12838). 확인하려면 먼저 COMMIT 해야 합니다

▪ 이 문제의 핵심

- 병렬 DML은 세션 활성화(**ENABLE PARALLEL DML**)가 전제입니다. 활성화 없이는 힌트가 있어도 DML이 직렬로 갈 수 있습니다.
- 병렬 DML로 수정한 객체는 같은 트랜잭션에서 재접근 불가입니다. 커밋/롤백 전에 조회·수정하면 ORA-12838이 납니다.

3. APPEND(Direct Path)는 HWM 위 새 블록에 기록하고, 병렬 INSERT는 PX 서버별 임시 세그먼트를 병합합니다.
4. NOLOGGING+Direct Path는 redo를 최소화해 빠르지만 미디어 복구 리스크가 있어 적재 후 백업이 필요할 수 있습니다.
 - 한 줄 정리 병렬 DML로 적재한 객체는 같은 트랜잭션에서 커밋 전 재조회가 막혀 ORA-12838이 나므로, "커밋 없이 곧바로 조회해 확인할 수 있다"는 것은 규칙을 뒤집은 진술입니다.

문제 16. 정답 ③

로컬 파티션 인덱스는 데이터 파티션과 1:1로 짝지어집니다. 다만 프루닝(불필요한 인덱스 파티션 건너뛰기)이 되려면 파티션 키 조건이 있어야 하고, 그 판단은 인덱스가 prefixed나 nonprefixed냐로 갈립니다.

[로컬 인덱스별 파티션 접근 범위]

ix_loc_pfx (발행일자, 공급자번호) → 선두가 파티션 키
 WHERE 발행일자 = ... → 파티션 프루닝 → 인덱스 파티션 1개
 ix_loc_npfx (거래처번호) → 선두가 파티션 키 아님
 WHERE 거래처번호 = ... → 어느 파티션인지 모름 → 인덱스 파티션 전부 스캔
 WHERE 거래처번호=... AND 발행일자=... → 그제야 프루닝 가능

③는 "로컬 인덱스"라는 부분적 사실을 딛고, nonprefixed 인덱스의 프루닝 효과가 prefixed와 "같다"고 단정합니다. 그러나 로컬이라는 성질만으로 프루닝이 보장되지는 않습니다. ix_loc_npfx는 선두 칼럼이 거래처번호라, 발행일자 조건이 없으면 특정 거래처의 행이 어느 분기 파티션에 흩어져 있는지 알 수 없어 모든 인덱스 파티션을 훑어야 합니다(②가 옳게 서술). 발행일자 조건이 함께 주어져야 비로소 prefixed처럼 단일·소수 파티션으로 좁혀집니다. 즉 "로컬"이라는 한 조건이 최선(1개 파티션)을 최악(전 파티션 스캔)으로 뒤집는 것을 무시한 부분진실 진술이 ③입니다. 그래서 부적절한 것은 ③이며, 이는 ②와 정면으로 모순됩니다.

①②④은 프리픽스드 로컬의 프루닝(①), nonprefixed의 전 파티션 스캔(②), DROP PARTITION 시 로컬 자동 정리 vs 글로벌 UNUSABLE(④)을 모두 옳게 서술합니다.

선택지	판정	이유
①	○	ix_loc_pfx는 선두가 파티션 키(발행일자)라, 날짜 조건이 오면 프루닝으로 대응 인덱스 파티션만 봄니다
②	○	ix_loc_npfx는 선두가 거래처번호라, 발행일자 조건이 없으면 어느 파티션인지 몰라 모든 인덱스 파티션을 스캔합니다
③	×	nonprefixed 로컬은 발행일자 조건이 없으면 전 인덱스 파티션을 훑으므로 프루닝 효과가 prefixed와 같지 않습니다. "로컬이니 단일 파티션"은 부분진실입니다
④	○	DROP PARTITION 시 로컬 인덱스 파티션은 함께 제거되지만, 글로벌 인덱스는 UNUSABLE이 되어 REBUILD(또는 UPDATE INDEXES)가 필요할 수 있습니다

▪ 이 문제의 핵심

1. 로컬 인덱스라는 사실만으로 파티션 프루닝이 보장되지 않습니다. 프루닝의 관건은 파티션 키 조건과 prefixed/nonprefixed 여부입니다.
2. 로컬 프리픽스드(발행일자, ...)는 파티션 키 조건이 오면 대응 인덱스 파티션만 탐색합니다.
3. 로컬 네프리픽스드(거래처번호)는 파티션 키 조건이 없으면 모든 인덱스 파티션을 스캔합니다 — 이때 프리픽스드와 효과가 다릅니다.
4. DROP PARTITION 시 로컬 인덱스 파티션은 자동 정리되지만, 글로벌 인덱스는 UNUSABLE이 되어 재생성이 필요할 수 있습니다.

▪ 한 줄 정리 네프리픽스드 로컬 인덱스는 파티션 키 조건이 없으면 전 인덱스 파티션을 스캔하므로, "로컬이니 단일 파티션만 보아 프리픽스드와 같다"는 것은 로컬이라는 부분진실로 최선을 최악으로 뒤집은 진술입니다.

문제 17. 정답 ③

분배 방식은 조인(Id 3)의 두 입력이 각각 어떤 PX SEND를 거치는지로 읽습니다.

Id 5 PX SEND HASH :TQ10000 (Q1,00) 처방전 쪽 생산자 → 조인 키 해시로 재분배
 Id 7 TABLE ACCESS FULL 처방전 (Q1,00)
 Id 9 PX SEND HASH :TQ10001 (Q1,01) 조제내역 쪽 생산자 → 조인 키 해시로 재분배
 Id 11 TABLE ACCESS FULL 조제내역 (Q1,01)
 Id 3 HASH JOIN (Q1,02) ← Id 4 · Id 8 PX RECEIVE로 양쪽을 받아 조인

처방전(Id 7)은 Id 5 PX SEND HASH로, 조제내역(Id 11)은 Id 9 PX SEND HASH로 양쪽 모두 조인 키(처방ID) 해시에 따라 재분배됩니다.

재분배된 두 스트림을 조인 소비자 집합(Q1,02)의 Id 4·Id 8 **PX RECEIVE**가 받아 Id 3 **HASH JOIN**에서 같은 해시 버킷 범위의 양쪽 행만 맞물려 조인합니다.

양쪽에 **PX SEND HASH**가 있는 이 방식이 hash-hash 분배입니다. 여기서 broadcast를 쓰지 않은 이유가 발문의 핵심입니다. broadcast는 한쪽 테이블을 모든 병렬 서버에 통째로 복제하는 방식이라, 소형이 아주 작을 때만 이득입니다. 그런데 이 조인은 작은 쪽(조제내역)도 8,900만 건으로 대형이라, 이를 서버 수만큼 복제하면 복제량이 폭증합니다. 그래서 각 입력을 한 번씩만 해시 재분배하는 hash-hash가 선택됩니다.

- ③ hash-hash : 처방전 = SEND HASH, 조제내역 = SEND HASH → Id 5·Id 9와 일치
- ② broadcast-none : 소형 = SEND BROADCAST, 대형 = 로컬 스캔 → BROADCAST 노드 없음
- ① partition-wise : 조인 입력에 PX SEND 없음 → Id 5·Id 9에 SEND HASH 있음
- ④ hash-broadcast : 한쪽 SEND HASH + 다른 쪽 SEND BROADCAST → 양쪽 다 SEND HASH라 불일치

조인 입력 양쪽에 **PX SEND HASH**가 나란히 있다는 사실이 broadcast·partition-wise 계열을 모두 배제합니다. 따라서 ③이 옳습니다.

선택지	판정	이유
①	×	파티션 와이즈면 조인 입력에 PX SEND가 없어야 합니다. Id 5·Id 9에 PX SEND HASH 가 있고 두 테이블은 파티셔닝돼 있지도 않으므로 파티션 쌍 로컬 조인이 아닙니다
②	×	broadcast-none이면 소형 쪽에 PX SEND BROADCAST 가 있어야 하는데 플랜에 BROADCAST 노드가 없습니다. 조제내역은 Id 9 PX SEND HASH 로 재분배됩니다
③	○	Id 5·Id 9가 모두 PX SEND HASH 로, 처방전·조제내역을 양쪽 다 처방ID 해시로 재분배합니다. 둘 다 대형이라 broadcast 대신 hash-hash를 쓴 상황과 일치합니다
④	×	hash-broadcast면 한쪽만 PX SEND HASH , 다른 쪽은 PX SEND BROADCAST 여야 합니다. Id 5·Id 9 둘 다 PX SEND HASH 라 분배 조합을 뒤바꾼 진술입니다

■ 이 문제의 핵심

- 조인 입력 양쪽에 **PX SEND HASH**가 있으면 hash-hash입니다. 두 테이블을 각각 조인 키 해시로 한 번씩 재분배해 같은 버킷끼리 조인합니다.
- broadcast는 소형 한쪽만 전체 복제합니다. 작은 쪽도 대형이면(8,900만 건) 서버 수만큼 복제 비용이 커져 broadcast가 불리하고, hash-hash가 유리합니다.
- 분배 방식은 PX SEND 종류로 판별합니다 — **SEND HASH**면 해시 재분배, **SEND BROADCAST**면 복제, 조인 입력에 SEND가 없으면 파티션 와이즈입니다.
- hash-hash·broadcast-none·hash-broadcast는 SEND 노드 구성이 서로 다릅니다. 이 플랜은 양쪽 다 **SEND HASH**라 나머지 조합이 모두 배제됩니다.

■ 한 줄 정리 조인 입력 처방전·조제내역이 Id 5·Id 9의 **PX SEND HASH**로 양쪽 다 처방ID 해시 재분배되어 Q1,02에서 같은 버킷끼리 조인하므로, 둘 다 대형이라 broadcast 대신 hash-hash를 택한 분배입니다.

문제 18. 정답 ①

소트 튜닝에서 인덱스가 지우는 것은 "정렬된 순서가 필요한" 소트입니다. 소트 계열 오퍼레이션이라고 다 같은 이유로 도는 게 아닙니다.

- 순서에 기대는 소트 : SORT ORDER BY, SORT GROUP BY, SORT UNIQUE(DISTINCT)
→ 인덱스가 그 순서를 미리 주면 생략 가능
- 순서와 무관한 소트 : SORT AGGREGATE (GROUP BY 없는 전체 집계)
→ 값을 훑어 누적할 뿐, 정렬 순서로 대체될 대상이 아님

①는 이 경계를 정확히 그었습니다. ORDER BY·GROUP BY는 인덱스의 정렬 순서를 그대로 쓰면 소트를 생략할 수 있지만, SORT AGGREGATE는 그룹 경계를 나눌 필요 없이 전체를 한 덩어리로 누적하는 연산이라 정렬 순서와 무관합니다. 따라서 "인덱스로 대체되는 소트"에 넣을 수 없습니다.

나머지는 일부 소트에만 성립하는 성질을 전체로 부풀린 하위항목 전체화 오류입니다.

②: "인덱스가 정렬을 주면 소트가 사라진다"는 순서 기반 소트에만 맞습니다. SORT AGGREGATE까지 싸잡아 대체된다고 넓힌 것이 오류입니다.

③: 디스크 소트(one-pass·multi-pass)는 수행 위치가 메모리에서 Temp로 바뀔 뿐, 오퍼레이션 종류를 SORT UNIQUE로 바꾸지 않습니다. SORT UNIQUE는 중복 제거용 한 종류일 뿐, 디스크 소트 전체의 이름이 아닙니다.

④: ①가 선을 그은 바로 그 지점을 반대로 넘었습니다. 전체 집계 SORT AGGREGATE는 정렬 순서를 이용해도 사라지지 않습니다.

선택지	판정	이유
①	○	ORDER BY·GROUP BY는 인덱스 정렬 순서로 소트를 생각할 수 있으나, 순서와 무관한 전체 집계 SORT AGGREGATE는 대체 대상이 아닙니다
②	×	인덱스로 대체되는 것은 순서에 기대는 소트뿐입니다. SORT AGGREGATE까지 대체된다고 넓힌 하위항목 전체화 오류입니다
③	×	디스크 소트는 수행 위치가 Temp로 바뀔 뿐 오퍼레이션 종류를 바꾸지 않습니다. SORT UNIQUE는 중복 제거용 한 종류이지 디스크 소트 전체의 이름이 아닙니다
④	×	SORT AGGREGATE는 그룹 경계 없이 전체를 누적하는 연산이라 정렬 순서와 무관하며, 인덱스 순서를 써도 사라지지 않습니다

▪ 이 문제의 핵심

1. 인덱스가 지우는 소트는 "순서가 필요한" 소트. SORT ORDER BY·GROUP BY·UNIQUE가 대상입니다.
2. SORT AGGREGATE는 예외. GROUP BY 없는 전체 집계는 정렬 순서에 기대지 않아 인덱스로 대체되지 않습니다.
3. 디스크 소트는 위치 문제지 종류 문제가 아니다. Sort Area를 넘으면 Temp로 갈 뿐, 오퍼레이션 이름이 SORT UNIQUE로 바뀌지 않습니다.
4. 부분 성질을 전체로 넓히지 말 것. "인덱스=소트 생략"은 순서 기반 소트에 한정된 이야기입니다.

▪ 한 줄 정리 인덱스의 정렬 순서로 생략되는 것은 ORDER BY·GROUP BY·UNIQUE 같은 순서 기반 소트뿐이고, 순서와 무관한 SORT AGGREGATE까지 대체된다고 넓히거나 디스크 소트를 SORT UNIQUE로 부르는 것은 부분을 전체로 부풀린 하위항목 전체화 오류입니다.

문제 19. 정답 ④

READ COMMITTED(락 기반)는 SELECT가 잡은 공유(S) 락을 읽은 직후 바로 해제합니다. 트랜잭션 끝까지 유지하지 않으므로, 같은 조건을 다시 읽는 사이에 다른 트랜잭션이 커밋하면 값이 달라질 수 있습니다 — 이것이 Non-Repeatable Read입니다.

[타임라인 - 잔여병상(W3) 초기 8]

t1 TX2 SELECT → (가) = 8 (S 락 잡고 바로 해제)
t2 TX1 UPDATE = 8 - 3 = 5 ; COMMIT (커밋 완료)
t3 TX2 SELECT → (나) = 5 (커밋된 새 값을 읽음)

(가)=8, (나)=5 → 같은 조건인데 값이 바뀜 = Non-Repeatable Read
(더티 아님: t3에서 읽은 5는 t2에서 이미 커밋된 값)

[격리수준을 REPEATABLE READ로 올리면]

t1 TX2 SELECT → 8 (S 락을 트랜잭션 끝까지 유지)
t2 TX1 UPDATE ... → TX2의 S 락에 막혀 대기(블록)
t3 TX2 SELECT → 8 ← 반복 읽기 보장 → (나)=8

READ COMMITTED에서 (가)=8, (나)=5로 재조회 값이 달라졌으니 Non-Repeatable Read가 맞습니다. 두 값 모두 커밋된 값을 읽었으므로 Dirty Read는 아닙니다. TX2를 REPEATABLE READ로 올리면 첫 읽기의 S 락이 트랜잭션 끝까지 유지되어 TX1의 UPDATE가 대기하고, TX2는 8을 반복해서 읽습니다. ④이 두 값과 현상, 격리수준 상향 효과를 모두 맞혔습니다.

선택지	판정	이유
①	×	READ COMMITTED의 S 락은 읽은 직후 해제되지 트랜잭션 끝까지 유지되지 않습니다. 그래서 (나)는 8이 아니라 5로 바뀝니다
②	×	t3에서 읽은 5는 t2에서 이미 커밋된 값이라 Dirty Read가 아닙니다. Dirty Read는 커밋 전 값을 읽는 현상입니다
③	×	두 값이 모두 커밋된 값이라는 사실은 RC·상위 어디서나 참이라 판별 근거가 못 됩니다. 값이 8→5로 달라졌으니 이상현상이 없다고 볼 수 없고 (가)는 8입니다
④	○	(가)=8, (나)=5로 재조회 값이 달라진 Non-Repeatable Read이고, REPEATABLE READ로 올리면 S 락 유지로 TX1이 대기해 (나)도 8이 됩니다

▪ 이 문제의 핵심

1. Non-Repeatable Read = 같은 조건의 재조회 결과가 트랜잭션 도중 달라지는 현상. 여기서는 8→5로 (가)≠(나)입니다.
2. READ COMMITTED의 S 락은 읽은 직후 해제되어 재조회 사이의 커밋을 막지 못합니다.
3. Dirty Read와의 구분: t3의 5는 이미 커밋된 값이므로 더티가 아닙니다. 더티는 커밋 전 값을 읽는 경우입니다.
4. REPEATABLE READ로 올리면 첫 읽기의 S 락이 유지되어 상대 UPDATE가 대기하므로 반복 읽기가 보장됩니다.

5. "두 값 모두 커밋된 값"은 RC·상위에 공통인 단서라 격리수준·이상현상의 판별 근거가 아닙니다.

- 한 줄 정리 락 기반 READ COMMITTED는 S 락을 즉시 해제하므로 (가)=8, (나)=5의 Non-Repeatable Read가 나며, REPEATABLE READ로 올려야 S 락 유지로 반복 읽기가 보장됩니다 — "커밋된 값을 읽었다"는 공통 사실만으로 이상현상 유무를 가릴 수는 없습니다.

문제 20. 정답 ④

TX 락 네 행을 두 세션으로 갈라 읽으면 방향이 한눈에 잡힙니다. 각 세션은 자신이 잠근 행의 트랜잭션 락을 X로 보유하면서(BLOCK=1), 상대가 잠근 행의 트랜잭션 락을 X로 REQUEST해 대기합니다.

```
SID 30 : TX 327681 · 14522 LMODE=6 BLOCK=1 → 1001 행 보유(막는 중)
        TX 393225 · 14870 REQUEST=6      → 2002 행 요청(대기)
SID 52 : TX 393225 · 14870 LMODE=6 BLOCK=1 → 2002 행 보유(막는 중)
        TX 327681 · 14522 REQUEST=6      → 1001 행 요청(대기)
```

30 → 52가 보유한 393225 를 기다림

52 → 30이 보유한 327681 를 기다림

= 서로가 서로를 기다리는 순환 대기(교착 상태, Deadlock)

④는 두 가지를 뒤집었습니다. 첫째, 표에는 두 세션 모두 BLOCK=1이면서 서로의 자원을 REQUEST하고 있어 명백한 양방향(순환) 대기인데, ④는 이를 "한쪽만 막는 단방향 블로킹"이라 했습니다. 둘째, 오라클은 교착을 자동으로 감지해 한 세션의 문장을 롤백(ORA-00060)하고 교착을 즉시 해소하는데, ④는 "감지하지 못해 무한정 대기한다"고 정반대로 서술했습니다. 그래서 부적절한 진술은 ④입니다. 반면 ①②③은 보유·요청 방향(①②)과 오라클의 자동 감지·문장 롤백(③)을 옳게 설명합니다.

선택지	판정	이유
①	○	SID 30은 327681·14522를 X(6)로 보유(BLOCK=1)하고 393225·14870을 REQUEST=6으로 대기합니다. 표의 두 행 그대로입니다
②	○	SID 52는 대칭으로 393225·14870 보유(BLOCK=1), 327681·14522 요청이라 두 세션이 서로를 기다리는 순환 대기가 성립합니다
③	○	오라클은 교착을 자동 감지해 한쪽 문장을 롤백하고 ORA-00060을 냅니다. 롤백 단위는 문장이지만 트랜잭션 전체가 아닙니다
④	×	두 세션이 모두 보유·요청을 엇갈리게 걸어 양방향 교착이며, 오라클은 이를 감지해 해소합니다. 단방향·미감지·무한대기는 모두 반대 진술입니다

▪ 이 문제의 핵심

1. 교착(Deadlock)은 두 세션이 서로가 보유한 자원을 맞요청할 때 성립합니다. v\$sqllock에서 양쪽이 BLOCK=1이면서 상대의 TX를 REQUEST하는 모습이 그 징표입니다.
2. TX 락은 (ID1·ID2)로 트랜잭션을 식별합니다. 30이 보유한 327681을 52가 REQUEST하고, 52가 보유한 393225를 30이 REQUEST해 고리가 닫힙니다.
3. 오라클은 교착을 자동 감지해 한 세션의 문장을 롤백(ORA-00060)하고 교착을 즉시 푼다 — 무한정 대기하지 않습니다.
4. 롤백 단위는 문장입니다. 교착이 감지돼도 해당 세션의 트랜잭션 전체가 사라지는 것은 아니며, 나머지는 재시도·커밋·롤백으로 이어갈 수 있습니다.

- 한 줄 정리 두 세션이 서로 보유한 행 락을 맞요청하는 양방향 순환 대기가 교착이며, 오라클은 이를 자동 감지해 한쪽 문장을 롤백(ORA-00060)하므로 "단방향이라 교착이 아니고 감지 못 해 무한 대기한다"는 ④가 부적절합니다.